Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Optimasi Pengisian Paket pada Truk Peti Kemas dengan Algoritma Local Search

Tahun Akademik 2026–2027



Disusun oleh:

Kelompok 37

Samuelson Dharmawan Tanuraharja – 13524001
Nicholas Wise Saragih Sumbayak – 13524037
Neswa Eka Anggara – 13524136

Laboratorium Inteligensi Artifisial
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 Latar Belakang . . . 3
1.2 Rumusan Persoalan dan Tujuan . . . 3
1.3 Entitas dan Atribut . . . 4
1.4 Ketentuan Representasi . . . 4
1.5 Batasan Persoalan . . . 5
1.6 Fungsi Objektif . . . 5

2 Dasar Teori **6**
2.1 Optimasi dan *Local Search* . . . 6
2.2 *State*, *Neighbor*, dan *Search Landscape* . . . 6
2.3 *Hill Climbing* . . . 7
2.4 *Simulated Annealing* . . . 8
2.5 *Genetic Algorithm* . . . 9

3 Pembahasan dan Implementasi **10**
3.1 Arsitektur Aplikasi . . . 10
3.1.1 Struktur Aplikasi . . . 10
3.1.2 Tech Stack . . . 11
3.2 Format Berkas Konfigurasi . . . 11
3.3 Pemodelan Problem dan Constraint . . . 12
3.3.1 Data Problem . . . 12
3.3.2 Representasi State dan Orientasi . . . 14
3.3.3 Validasi Constraint . . . 14
3.3.4 Objective Function . . . 16
3.4 Implementasi Algoritma *Local Search* . . . 17
3.4.1 Inisialisasi State dan Pembangkitan Neighbor . . . 17
3.4.2 Hill Climbing . . . 17
3.4.3 Simulated Annealing . . . 20
3.4.4 Genetic Algorithm . . . 20

4 Eksperimen dan Analisis **22**
4.1 Rancangan Eksperimen . . . 22
4.1.1 Data Uji dan Lingkungan Eksperimen . . . 22
4.1.2 Konfigurasi Parameter dan Pengulangan Eksperimen . . . 22
4.2 Hasil Eksperimen dan Visualisasi Program . . . 23
4.2.1 Hill Climbing . . . 23
4.2.2 Simulated Annealing . . . 24
4.2.3 Genetic Algorithm . . . 25
4.3 Analisis Eksperimen . . . 25
4.3.1 Kedekatan terhadap Optimum Global . . . 26
4.3.2 Perbandingan Hasil Pencarian Antaralgoritma . . . 27
4.3.3 Perbandingan Durasi Pencarian . . . 28
4.3.4 Konsistensi Hasil Akhir . . . 28
4.3.5 Analisis Genetic Algorithm pada Konfigurasi Tetap . . . 29

5 Kesimpulan dan Saran **31**
5.1 Kesimpulan . . . 31
5.2 Saran . . . 31

6 Pembagian Tugas Tiap Anggota Kelompok **31**

Daftar Pustaka **31**

A Form Penggunaan Generative AI **31**

B Lampiran Lainnya (Bila Ada) 31

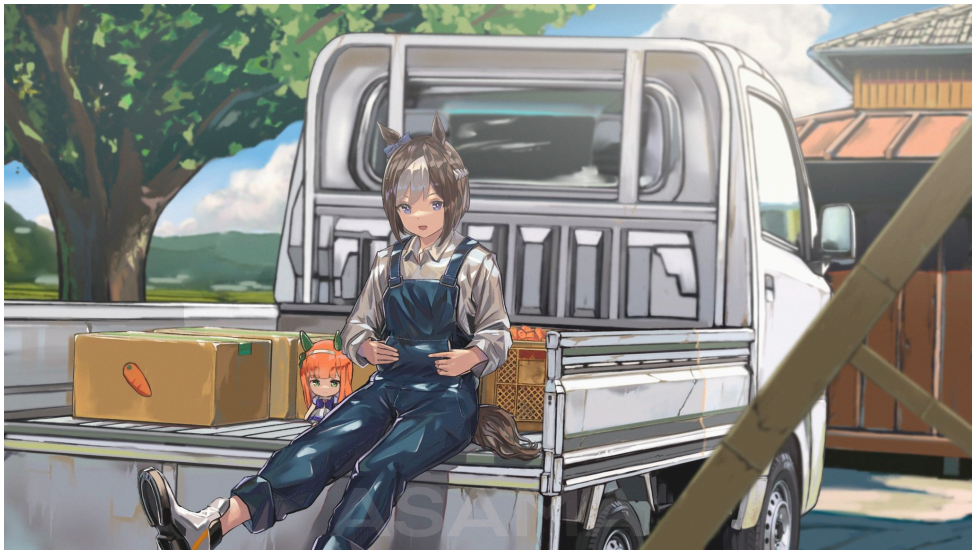
C Bukti Pengujian Manual 32

1 Pendahuluan

1.1 Latar Belakang

Saat menyiapkan pengiriman Mejiro Express, Special Week menghadapi paket dengan ukuran, nilai, berat, dan tingkat kerapuhan yang berbeda-beda. Persoalannya adalah menentukan paket yang dapat dimuat ke dalam truk peti kemas beserta penempatannya dalam ruang tiga dimensi. Paket bernilai tinggi menarik untuk dibawa, tetapi ruang peti kemas dan kapasitas angkut truk terbatas. Selain itu, susunan harus mencegah paket bertumpang tindih atau melayang, dan paket rapuh tidak boleh menjadi penopang paket lain [1].

Keputusan pemilihan dan penempatan saling berkaitan. Memuat suatu paket mengurangi ruang tersisa dan menambah berat muatan, sedangkan posisi dan orientasinya memengaruhi kemungkinan menempatkan paket lain dengan memenuhi batasan keselamatan. Oleh sebab itu, persoalan ini mencakup pemilihan paket sekaligus penyusunan seluruh paket yang dimuat, sementara paket yang tidak terpilih tetap berada di luar peti kemas.



Gambar 1. Mejiro Express sebagaimana ditunjukkan dalam spesifikasi (Sumber: 真夏の銀堡垒🐴C108土曜日 東地区 ホ-10a on X).

1.2 Rumusan Persoalan dan Tujuan

Persoalan menerima paket berbentuk balok atau kubus serta peti kemas dengan ukuran dan kapasitas angkut tertentu. Setiap paket dapat dipilih untuk dimuat atau dibiarkan di luar, dan paket yang dimuat perlu ditempatkan dengan posisi serta orientasi yang sesuai. Tujuannya adalah memperoleh susunan yang memenuhi batas ruang, kapasitas berat, dan aturan penopang, dengan total nilai paket termuat sebesar mungkin [1].

1.3 Entitas dan Atribut

Masukan persoalan terdiri atas paket dan sebuah truk. Tabel 1 merangkum atribut yang diperlukan; contoh angka hanya ilustrasi, bukan data fixture maupun hasil eksperimen.

Entitas	Atribut	Contoh	Keterangan
Paket	ID	`RusdiPackage`	Identitas paket.
Paket	Dimensions	$\{w:3, l:4, h:1\}$	Lebar w (*width*), panjang l (*length*), dan tinggi h (*height*) paket.
Paket	Orientation	$\{w:x, l:y, h:z\}$	Enam pemetaan sumbu yang sejajar dengan sisi paket. Rotasi 90° terhadap z mengubah contoh menjadi $\{w:y, l:x, h:z\}$.
Paket	Value	60	Nilai paket dalam satuan unit.
Paket	Weight	5	Berat dalam satuan berat.
Paket	IsFragile	`False`	Boolean; `False` berarti tidak rapuh.
Paket	ETA	3	Perkiraan sisa hari hingga kedatangan (*Estimated Time of Arrival*).
Truk	Dimensions	$\{w:10, l:20, h:5\}$	Lebar w, panjang l, dan tinggi h peti kemas.
Truk	MaxCapacity	500	Kapasitas maksimum dalam satuan berat.

Tabel 1. Entitas, atribut, dan contoh ilustratif persoalan.

1.4 Ketentuan Representasi

Paket dan peti kemas berbentuk balok atau kubus dengan dimensi yang dapat berbeda. Secara konseptual, data dapat dinyatakan sebagai pasangan kunci-nilai seperti JSON, tanpa membatasi struktur data. State mencakup semua paket: paket termuat berada seluruhnya di dalam peti kemas, sedangkan yang tidak termuat berada seluruhnya di luar. Posisi dinyatakan dengan koordinat bilangan bulat tiga dimensi. Orientasi memiliki enam permutasi dimensi, yang diperoleh melalui rotasi 90° terhadap sumbu x, y, atau z. State awal dibangkitkan secara acak.

Langkah perubahan (*move*) untuk membentuk tetangga (*neighbor*) pada setiap iterasi meliputi:

1. Menukar penempatan dua paket, baik keduanya di dalam, keduanya di luar, maupun satu di dalam dan satu di luar peti kemas.
2. Memindahkan paket ke koordinat yang berbeda.
3. Merotasi paket 90° terhadap sumbu x, y, atau z.

Representasi solusi akhir (*final state*) memetakan seluruh paket ke status, posisi, dan orientasinya, serta menentukan keterisian setiap koordinat dalam peti kemas. State dan visualisasi harus menunjukkan:

1. Keadaan seluruh paket, baik yang termuat maupun yang berada di luar.
2. Keadaan peti kemas pada truk.
3. Total nilai paket pada state tersebut.

Ketentuan ini tidak menetapkan struktur penyimpanan tertentu untuk pemetaan tersebut [1].

1.5 Batasan Persoalan

Untuk paket termuat i, nyatakan sudut kiri-bawah-belakang sebagai (x_i, y_i, z_i) dan dimensi efektif sebagai (w_i, l_i, h_i). Alas setiap paket termuat adalah

$$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + l_i\}. \tag{1}$$

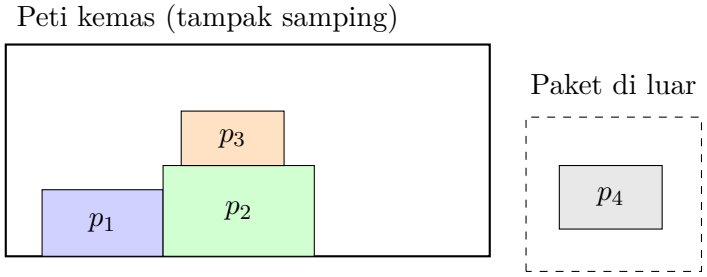
Seluruh volume paket termuat harus berada di dalam batas truk. Paket-paket tidak boleh berbagi volume; dua paket boleh bersentuhan pada batas permukaannya. Titik alas memiliki penopang (*support*) apabila $z_i = 0$, yaitu berada di lantai truk, atau permukaan atas paket lain berada tepat di bawah titik tersebut. Paket disebut melayang apabila tidak ada satu pun titik alas yang memiliki penopang. Dengan demikian, dukungan parsial cukup; seluruh alas tidak harus ditopang. Paket rapuh tidak boleh menjadi penopang paket lain, dan jumlah berat paket termuat tidak boleh melebihi kapasitas truk [1]. Himpunan $\mathcal{F}$ menyatakan seluruh susunan yang memenuhi batasan tersebut.

1.6 Fungsi Objektif

Untuk satu truk, tujuan persoalan adalah memaksimalkan total nilai paket termuat di antara susunan yang sah:

$$\max_{s \in \mathcal{F}} f(s), \qquad f(s) = \sum_{i \text{ termuat pada } s} \text{Value}_i. \tag{2}$$

Paket di luar peti kemas tidak berkontribusi terhadap nilai tersebut. ETA merupakan atribut paket, tetapi tidak menjadi bobot pada fungsi objektif utama [1].



Gambar 2. Ilustrasi konseptual paket termuat dan paket di luar peti kemas. Susunan sebenarnya ditentukan dalam tiga dimensi.

2 Dasar Teori

2.1 Optimasi dan *Local Search*

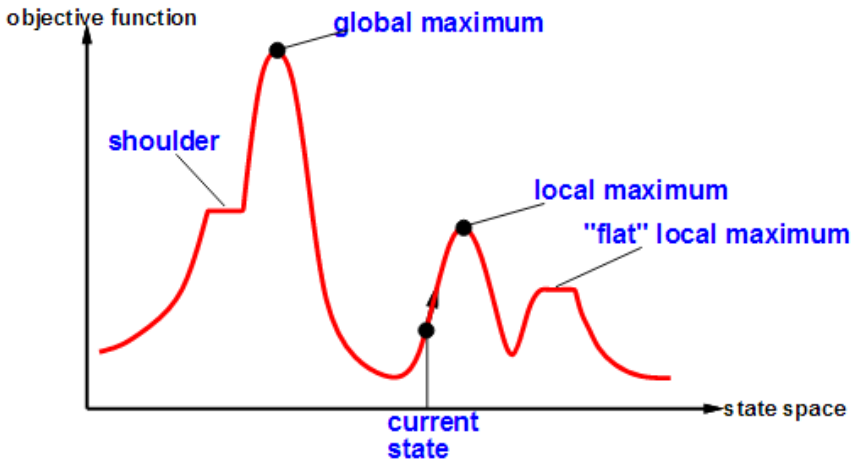
Dalam optimasi, *objective function* memberi nilai pada setiap kandidat solusi, dan pencarian diarahkan pada kandidat bernilai tinggi. Jika terdapat ***constraint*** (batasan), kandidat yang dipertimbangkan termasuk dalam himpunan yang memenuhi batasan tersebut. Dengan $\mathcal{F}$ sebagai himpunan kandidat yang sah, tujuan umum masalah maksimisasi dapat ditulis $\max_{s \in \mathcal{F}} f(s)$, dengan $f(s)$ sebagai nilai kandidat s [2].

Local search bekerja pada satu kandidat lengkap dan memperbaikinya melalui perubahan bertahap. Nilai solusi ditentukan oleh konfigurasi akhir, bukan urutan perubahan yang membentuknya. Pendekatan ini sesuai ketika ruang kemungkinan terlalu besar untuk diperiksa satu per satu [2, 3].

2.2 *State*, *Neighbor*, dan *Search Landscape*

State (keadaan) adalah kandidat solusi yang lengkap. Pencarian berawal dari *initial state* (state awal), yang dapat dibangkitkan secara acak. Sebuah *move* (langkah perubahan) menghasilkan ***neighbor*** (tetangga), yaitu state yang berbeda pada sebagian komponennya. Kumpulan neighbor yang dapat dicapai menentukan arah perpindahan dari state saat ini [3].

Nilai setiap state dapat dibayangkan sebagai ketinggian dalam *search landscape* (lanskap pencarian). ***Local maximum*** (puncak lokal) bernilai setidaknya sama tinggi dengan seluruh neighbor-nya, walaupun state lain mungkin memiliki nilai lebih tinggi. Nilai tertinggi di seluruh ruang pencarian disebut *global maximum* (puncak global). *Plateau* (daerah datar) merupakan kumpulan state bertetangga dengan nilai yang sama. Puncak lokal dan plateau menjelaskan mengapa pencarian yang hanya menerima kenaikan nilai dapat berhenti sebelum mencapai nilai tertinggi keseluruhan [2, 3].



Gambar 3. *Search landscape* dengan global maximum, local maximum, dan plateau. Diambil dari salindia *Beyond Classical Search* [3].

2.3 *Hill Climbing*

Seperti terlihat pada Gambar 3, pencarian yang hanya memilih kenaikan nilai dapat berhenti di local maximum. ***Hill climbing*** memulai dari satu state, membandingkannya dengan neighbor, lalu berpindah menurut aturan varian yang digunakan. Pada *steepest-ascent hill climbing*, versi teoretis mengevaluasi seluruh neighbor dan memilih yang memiliki nilai tertinggi, selama nilainya melebihi state saat ini. Jika beberapa neighbor sama-sama terbaik, pemilihan salah satunya tidak mengubah nilai yang dicapai pada langkah itu.

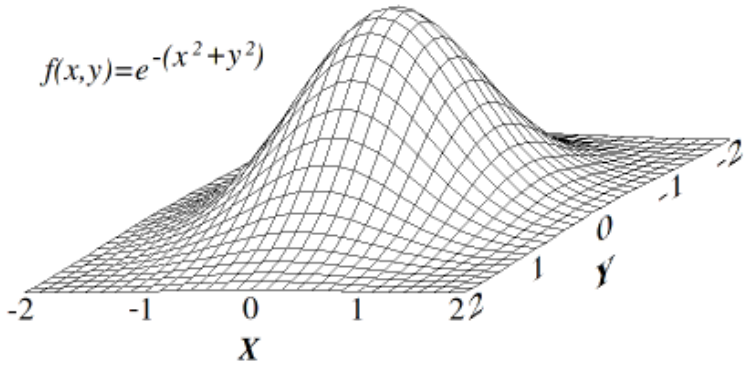
Dalam implementasi, seluruh neighborhood tidak diperiksa. Algoritma membangkitkan sampel terbatas neighbor yang sah, lalu memilih kandidat bernilai tertinggi dari sampel tersebut. Karena itu, **pemeriksaan terbatas pada sampel** berarti tidak ditemukannya peningkatan hanya menunjukkan bahwa sampel saat itu tidak memuat kandidat yang lebih baik; keadaan ini belum memastikan semua neighbor sudah diperiksa. Steepest-ascent menghindari langkah yang menurunkan atau mempertahankan skor, sehingga pencarian dapat berakhir di puncak lokal atau berhenti saat sampelnya tidak menunjukkan kenaikan [2, 3].

Stochastic hill climbing memilih secara acak dari neighbor yang meningkatkan skor. Pemilihan seragam memberi setiap kandidat peningkat peluang yang sama, sedangkan skema berbobot dapat memberi peluang lebih besar kepada kenaikan yang lebih besar. Implementasi mengambil satu neighbor acak per iterasi dan menerimanya jika skornya meningkat ketat. Jika kandidat tidak meningkatkan skor, state tetap dipertahankan dan pencarian melanjutkan iterasi hingga batas yang ditentukan. Kebijakan ini tidak menerima langkah setara untuk melintasi plateau [2, 3].

Hill climbing with sideways move juga menerima neighbor yang skornya sama, sehingga dapat bergerak di antara state pada plateau dan mungkin mencapai state yang memiliki peningkatan di sisi lain. Langkah setara tidak menaikkan skor, dan tanpa batas dapat membuat pencarian berputar atau bergerak lama tanpa kemajuan. Karena itu, **jumlah langkah setara berturut-turut dibatasi**. Langkah yang meningkatkan skor mengatur ulang hitungan tersebut; setelah batas tercapai, langkah setara berikutnya ditolak, sementara peningkatan ketat tetap dapat diterima.

Random-restart hill climbing mengulang steepest-ascent dari state awal berbeda untuk mencoba wilayah pencarian lain. Implementasi menjalankan satu climb dari state awal yang diberikan, lalu melakukan 10 restart, sehingga terdapat 11 climb. Setiap climb memiliki anggaran iterasinya sendiri; total pekerjaan dapat lebih besar daripada satu climb, tetapi jumlah restart dan sampel tetap membatasi bagian ruang yang diperiksa. Hasil terbaik dipilih dari seluruh climb, sedangkan state akhir adalah hasil climb terakhir. Restart dapat mengurangi ketergantungan pada satu puncak lokal, tetapi tidak memastikan optimum global; pencarian tetap tidak mengetahui solusi terbaik yang belum dikunjungi [2, 3].

Gambar 4 memperlihatkan fungsi dua variabel dengan satu puncak. Pada masalah maksimisasi, perpindahan ke nilai yang lebih tinggi dapat dibayangkan sebagai langkah menaiki permukaan tersebut. Susunan paket merupakan state diskret dengan banyak constraint, sehingga lanskapnya tidak harus sehalus ilustrasi ini; puncak lokal dan plateau tetap menjadi hambatan seperti pada Gambar 3.



Gambar 4. Ilustrasi permukaan objektif untuk hill climbing. Sumber: Wikimedia Commons, *Hill climb.png*.

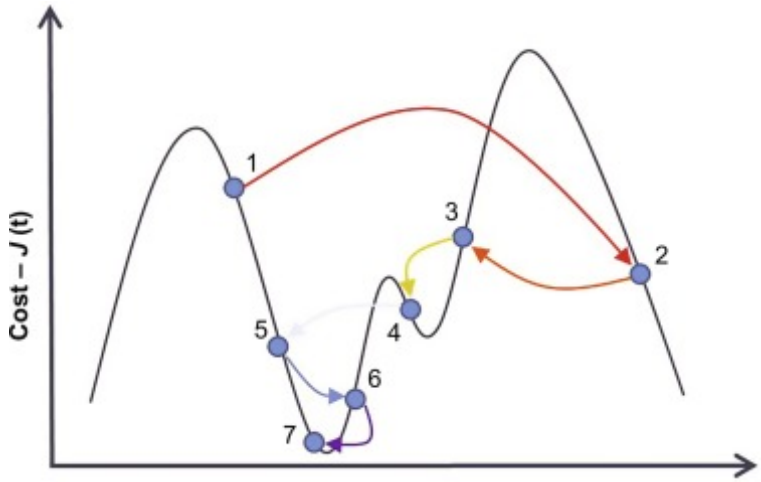
2.4 *Simulated Annealing*

Hill climbing tidak selalu bisa keluar dari local maximum dengan move yang hanya memperbaiki nilai. ***Simulated annealing*** sesekali menerima move yang menurunkan nilai agar pencarian dapat berpindah ke daerah lain. Peluang menerima penurunan ditentukan oleh *temperature* (temperatur). Untuk $\Delta f = f(s') - f(s) < 0$ dan $T > 0$, peluang menerima neighbor s' adalah

$$\Pr(\text{accept } s') = \exp\left(\frac{\Delta f}{T}\right). \tag{3}$$

Pada temperature tinggi, algoritma lebih sering menerima penurunan nilai sehingga dapat keluar dari local maximum. Seiring temperature turun, peluang itu mengecil. *Cooling schedule* (jadwal penurunan temperatur) menentukan seberapa cepat perilaku pencarian berubah. Jadwal yang terlalu cepat dapat membuat pencarian kembali terjebak sebelum sempat menjelajahi state lain [2, 3].

Gambar 5 menunjukkan perpindahan pada lanskap biaya, termasuk langkah yang sementara menaikkan biaya sebelum pencarian mencapai daerah yang lebih rendah. Ilustrasi ini memakai sudut pandang minimisasi. Untuk persoalan pemuatan yang memaksimalkan $f(s)$, biaya dapat dinyatakan sebagai $E(s) = -f(s)$, sehingga kenaikan biaya pada gambar setara dengan penurunan skor objektif yang sesekali diterima oleh SA.

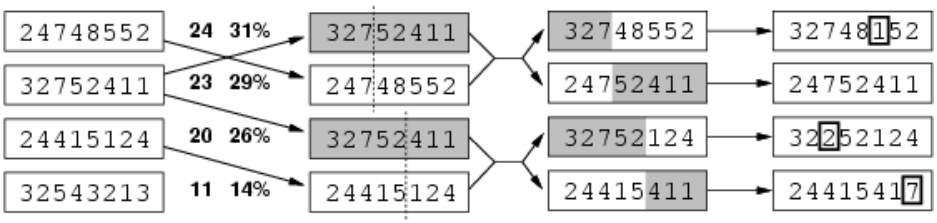


Gambar 5. Ilustrasi perpindahan simulated annealing pada lanskap biaya. Sumber: Elsevier, ilustrasi 2.14.

2.5 *Genetic Algorithm*

Jika hill climbing dan simulated annealing mengubah satu state dalam satu lintasan pencarian, ***genetic algorithm*** bekerja dengan *population* (populasi) yang berisi sejumlah *individual* (kandidat solusi). Setiap individual dinilai memakai *fitness function* (fungsi penilaian). Melalui *selection* (pemilihan induk), individual yang lebih baik mendapat peluang lebih besar untuk menghasilkan keturunan [2, 3].

Generasi baru dibentuk melalui *crossover* (penggabungan bagian dari dua induk) dan *mutation* (perubahan acak pada individual). Crossover mewariskan bagian dari solusi yang sudah ada, sementara mutation membantu menjaga keragaman population. Ukuran population dan banyaknya *generation* (generasi) memengaruhi jumlah kandidat yang diperiksa serta waktu pencarian. Bila suatu operator menghasilkan kandidat yang melanggar constraint, kandidat itu perlu ditangani sebelum dipakai sebagai solusi [2, 3].



Gambar 6. Alur selection, crossover, dan mutation pada genetic algorithm. Diambil dari salindia *Beyond Classical Search* [3].

3 Pembahasan dan Implementasi

3.1 Arsitektur Aplikasi

Pengguna mengubah paket dan truk melalui formulir web yang dimulai dari fixture armada bawaan di `src/frontend/src/mocks/fleetFixture.ts`. Configuration Manager juga mendukung impor dan ekspor berkas konfigurasi.

Konfigurasi diperiksa melalui endpoint `POST /api/problems/validate`. Endpoint ini memvalidasi model dan batas masukan, bukan susunan paket. Ketika run dimulai, frontend memeriksa parameter lalu mengirim problem, algoritma, seed, objektif, parameter, dan pilihan riwayat ke `POST /api/experiments/run`. Backend memvalidasi skema dan anggaran eksekusi, membangkitkan state awal, menjalankan algoritma, lalu memeriksa kelayakan state awal, akhir, dan terbaik serta kecocokan skornya sebelum membentuk respons.

Run dari UI tidak menyimpan hasil secara default. Pengguna harus mengaktifkan *Save run* pada panel setup untuk menulis hasil ke direktori keluaran. Request run maupun batch tanpa pilihan simpan juga tidak menulis berkas. Daftar run tersimpan dimuat dan dapat diperbarui melalui UI, serta tersedia melalui `GET/api/experiments`. Endpoint preset GA dan batch tersedia melalui `GET/api/experiments/presets/genetic_algorithm` dan `POST/api/experiments/batch`. Problem yang sedang diedit untuk run berikutnya terpisah dari problem asli milik run yang sedang ditampilkan.

3.1.1 Struktur Aplikasi

```
.
|-- data/
|-- output/
|-- test/
|-- src/
    |-- backend/
    | |-- app/
    | |-- api/
    | |-- domain/
    | |-- experiments/
    | |-- optimization/
    |-- frontend/
        |-- src/
            |-- api/
            |-- charts/
            |-- modals/
            |-- panels/
            |-- scene/
            |-- shell/
            |-- stores/
            |-- mocks/
```

Di backend, `domain` mendefinisikan data, `optimization` memuat validator dan algoritma, sedangkan `experiments` mengatur eksekusi, ringkasan, dan pencatatan hasil. Di frontend, `api` menangani request serta pemeriksaan respons; `charts`, `modals`, `panels`, `scene`, dan `shell` memisahkan grafik, dialog, panel, visualisasi tiga dimensi, dan kontrol antarmuka. Fixture armada berada di `src/frontend/src/mocks`; `stores` memuat state eksperimen aktif serta replay, sedangkan daftar run tersimpan dikelola di aplikasi.

Dataset uji yang dipakai untuk GA berada di `test/testing-dataset.json`. Run individual yang sengaja disimpan ditulis sebagai `output/<id>.json`; batch sebagai `output/<id>.batch.json` dan `output/<id>.batch.csv`. Tiga berkas GA untuk laporan diberi nama `output/ga-seed-42.json`, `output/ga-seed-123.json`, dan `output/ga-seed-2026.json`; masing-masing memuat konfigurasi, metrik, serta state awal, akhir, dan terbaik lengkap.

3.1.2 Tech Stack

Backend ditulis dengan **Python 3.12**. **FastAPI** menyediakan endpoint untuk memvalidasi problem, menjalankan eksperimen tunggal atau batch, dan membaca hasil yang tersimpan. Model **Pydantic** memeriksa tipe serta rentang nilai saat request diterima. Sesudah itu, modul optimasi menjalankan aturan yang sama untuk semua algoritma. **NumPy** digunakan untuk occupancy grid tiga dimensi, sehingga pemeriksaan ruang terisi dapat dilakukan pada irisan sel paket.

Frontend dibangun dengan **React** dan **TypeScript**. Komponen React memisahkan form problem, kontrol pencarian, visualisasi, dan ringkasan hasil. TypeScript membantu menjaga bentuk request dan respons tetap sesuai kontrak backend. **Vite** dipakai untuk menjalankan frontend selama pengembangan dan menghasilkan berkas aplikasi saat build.

Untuk menampilkan hasil, **React Three Fiber** dan **Three.js** menggambar peti kemas serta posisi paket dalam ruang tiga dimensi. **Recharts** menampilkan perubahan skor selama iterasi. **Zustand** menyimpan state eksperimen aktif serta indeks iterasi untuk replay.

3.2 Format Berkas Konfigurasi

Berkas konfigurasi memakai envelope JSON berversi yang menyimpan nama konfigurasi dan problem truk-paket; format ekspor yang sama digunakan untuk impor melalui Configuration Manager dan `POST /api/configs`. Field `description` opsional. UI juga menerima problem JSON langsung dengan `trucks` dan `packages`, lalu mengemasnya sebagai konfigurasi dengan nama dari berkas.

Cuplikan 1. Contoh envelope berkas konfigurasi JSON

```
{
  "schema_version": 1,
  "name": "Fleet example",
  "description": "Optional note",
  "problem": {
    "trucks": [{
      "id": "truck-1", "width": 10, "length": 20,
      "height": 8, "max_capacity": 1000
    }],
    "packages": [{
      "id": "package-1", "width": 2, "length": 3,
      "height": 1, "value": 10, "weight": 5,
      "is_fragile": false, "eta": 1
    }]
  }
}
```

Constraints dan penanganan kesalahan:

- Field wajib: `name` dan `problem`. Deskripsi opsional. Nilai `schema_version` hanya 1 dan menjadi default jika tidak diisi. Field tambahan pada envelope request API ditolak.
- Nama setelah trim tidak boleh kosong dan maksimal 80 karakter; deskripsi maksimal 500 karakter.
- Problem memerlukan 1–20 truk dan 0–100 paket. ID unik per jenis entitas dan maksimal 128 karakter.
- Dimensi harus bilangan bulat positif; `value`, `weight`, dan `max_capacity` finite dan tidak negatif; `eta` bilangan bulat tidak negatif; `is_fragile` boolean JSON.
- Total volume truk dibatasi 100.000 sel dan ukuran request 256 KiB.
- JSON rusak ditolak UI sebelum dikirim; validasi skema atau field di backend menghasilkan HTTP 422 jika gagal. Nama konfigurasi duplikat (tanpa membedakan kapitalisasi) menghasilkan 409; konfigurasi yang tidak ditemukan 404, dan konfigurasi bawaan tidak dapat dihapus (403).
- Konfigurasi valid belum berarti susunan feasible: berkas tidak memerlukan koordinat atau orientasi. Pencarian membentuk placement kemudian, dan validator state memeriksanya secara terpisah. Mode algoritma dan seed bukan bagian payload konfigurasi; parameter pencarian diatur pada panel setup.

3.3 Pemodelan Problem dan Constraint

Pemodelan dimulai dari data paket dan truk. Dari data itu, program membentuk state yang mencatat posisi setiap paket dan memeriksa constraint-nya. HC/SA hanya menilai neighbor yang sah. Pada GA, offspring tidak sah juga dievaluasi, tetapi penalti fitness memengaruhi seleksi. State awal, akhir, dan terbaik yang dikembalikan tetap feasible; skor hasil merupakan objective mentah.

3.3.1 Data Problem

Sesuai atribut konseptual pada Subbab 1.3, `Problem` menyimpan daftar `PackageSpec` dan `TruckSpec`. Dimensi lebar, panjang, dan tinggi diratakan menjadi field `width`, `length`, dan `height`; `IsFragile` menjadi `is_fragile`, dan `MaxCapacity` menjadi `max_capacity`. Atribut `Value`, `Weight`, dan `ETA` masing-masing disimpan sebagai `value`, `weight`, dan `eta`. Truk memiliki `id` sebagai identitas implementasi tambahan. Dukungan beberapa truk merupakan perluasan dari persoalan satu truk; setiap paket ditugaskan ke paling banyak satu truk. Cuplikan 2 hingga 4 memperlihatkan hubungan ketiga kelas tersebut.

Validasi masukan dilakukan sejak model dibuat. Dimensi harus berupa bilangan bulat positif, sedangkan nilai, berat, ETA, dan kapasitas tidak boleh negatif maupun tak hingga. `Problem` menolak ID paket atau truk yang berulang dan mensyaratkan setidaknya satu truk. Karena `trucks` berupa daftar, struktur yang sama dapat dipakai untuk satu truk maupun beberapa truk. Setelah data ini tersedia, penempatan setiap paket dicatat dalam state.

Cuplikan 2. Atribut paket pada PackageSpec

```
class PackageSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    value: float = Field(ge=0)
    weight: float = Field(ge=0)
    is_fragile: bool
    eta: int = Field(ge=0)
```

Cuplikan 3. Atribut truk pada TruckSpec

```
class TruckSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    max_capacity: float = Field(ge=0)
```

Cuplikan 4. Daftar paket dan truk dalam Problem

```
class Problem(BaseModel):
    packages: list[PackageSpec]
    trucks: list[TruckSpec]

    @model_validator(mode="after")
    def validate_ids_and_trucks(self) -> "Problem":
        if not self.trucks:
            raise ValueError("at least one truck is required")
        if len({package.id for package in self.packages}) != len(self.packages):
            raise ValueError("package IDs must be unique")
        if len({truck.id for truck in self.trucks}) != len(self.trucks):
            raise ValueError("truck IDs must be unique")
        return self
```

3.3.2 Representasi State dan Orientasi

Kelas `State` berisi daftar `PackagePlacement`, sesuai Subbab 1.4. Pada state yang sah, setiap paket memiliki tepat satu placement berisi ID paket dan truk, `Position`, serta `Orientation`. Orientasi disimpan pada placement, bukan `PackageSpec`. Posisi menunjuk sudut kiri-bawah-belakang (x mengikuti lebar, y panjang, z tinggi). Paket di luar memiliki `truck_id` dan `position` bernilai `None`. Posisi dan orientasi pada placement, bersama dimensi pada `PackageSpec`, menentukan pemetaan spasial; grid keterisian dibentuk saat validasi, bukan diserialisasi sebagai bagian terpisah dari `State`.

Cuplikan 5. Model penempatan paket

```
class PackagePlacement(BaseModel):
    package_id: str
    truck_id: str | None = None
    position: Position | None = None
    orientation: Orientation = Orientation.WLH

    @model_validator(mode="after")
    def validate_assignment(self) -> "PackagePlacement":
        if (self.truck_id is None) != (self.position is None):
            raise ValueError("truck_id and position must both be set or both be None")
        return self
```

Enam pilihan `Orientation` didefinisikan pada `app/domain/package.py`. Fungsi `get_oriented_dimensions` mengubah orientasi menjadi dimensi efektif (w_i, l_i, h_i) tanpa mengubah dimensi asli paket. Untuk kubus atau balok dengan sisi yang sama, beberapa orientasi menghasilkan dimensi efektif identik. Fungsi `get_unique_orientations` menghapus pilihan duplikat tersebut saat orientasi akan diacak. State yang terbentuk kemudian diperiksa sebagai satu kesatuan, bukan per paket saja.

3.3.3 Validasi Constraint

Fungsi `validate_state` pada `app/optimization/constraints.py` menjadi **pemeriksa akhir setiap kandidat** terhadap batasan Subbab 1.5, termasuk alas B_i pada Persamaan 1 dan dukungan parsial. Pemeriksaan dimulai dari kelengkapan assignment. Setiap paket dalam `Problem` harus muncul sekali dalam `State`, sedangkan ID paket dan truk yang tidak dikenal ditolak. Paket yang berada di luar tidak memakai ruang maupun kapasitas truk.

Validator membuat *occupancy grid* (grid keterisian) NumPy bagi setiap truk. Bentuk grid mengikuti (W_t, L_t, H_t), dengan sel kosong bernilai -1 dan sel terisi menyimpan indeks paket. Jika paket i ditempatkan di truk t pada (x_i, y_i, z_i) dengan dimensi efektif (w_i, l_i, h_i), seluruh bagian paket harus memenuhi

$$\begin{aligned} 0 \le x_i, \quad x_i + w_i \le W_t, \\ 0 \le y_i, \quad y_i + l_i \le L_t, \\ 0 \le z_i, \quad z_i + h_i \le H_t. \end{aligned} \tag{4}$$

Dalam kode, `_collect_occupancies` membandingkan posisi dan dimensi paket dengan ukuran grid. Cuplikan 6 menunjukkan pemeriksaan paket yang melewati dinding peti kemas. Setelah lolos, irisan sel yang akan ditempati diperiksa lagi. Cuplikan 7 menolak

paket ketika irisan tersebut sudah berisi indeks paket lain. Karena grid dibuat per truk, paket pada truk berbeda tidak saling bertabrakan.

Cuplikan 6. Pemeriksaan batas peti kemas

```
if any(
    start < 0 or start + size > limit
    for start, size, limit in zip(origin, dimensions, grid.shape, strict=True)
):
    errors.append(
        f"Package {placement.package_id} is out of bounds in truck {truck_id}"
    )
    unsafe_trucks.add(truck_id)
    continue
```

Cuplikan 7. Pemeriksaan overlap pada occupancy grid

```
for placement, package_index, region in regions:
    truck_id = placement.truck_id
    assert truck_id is not None
    cells = occupancies[truck_id][region]
    if np.any(cells != -1):
        errors.append(
            f"Package {placement.package_id} overlaps another in truck {truck_id}"
        )
        unsafe_trucks.add(truck_id)
        continue
    cells[...] = package_index
```

Paket pada $z_i = 0$ ditopang oleh lantai truk. Untuk paket di atas lantai, validator memeriksa sel tepat di bawah alasnya, yakni pada $z_i - 1$. Paket dinyatakan **melayang** jika tidak ada satu pun sel alas yang mendapat support dari paket lain. Jika ada support tetapi salah satu paket penopang bersifat rapuh, susunan juga ditolak. Pemeriksaan ini mengikuti definisi support pada tugas, yaitu cukup ada satu titik alas yang ditopang. Validator tidak mensyaratkan seluruh alas paket bersentuhan dengan penopang. Bagian inti pemeriksaan tersebut tampak pada Cuplikan 8.

Cuplikan 8. Pemeriksaan support dan paket rapuh

```
w, l, _ = get_oriented_dimensions(
    packages[placement.package_id], placement.orientation
)
x, y, z = position.x, position.y, position.z
below = occupancies[truck_id][x : x + w, y : y + l, z - 1]
supporters = np.unique(below[below >= 0])
if supporters.size == 0:
    support_errors.append(
        f"Package {placement.package_id} is floating in truck {truck_id}"
    )
elif any(problem.packages[int(index)].is_fragile for index in supporters):
    fragile_errors.append(
        f"Package {placement.package_id} rests on fragile support "
        f"in truck {truck_id}"
    )
```

Setelah posisi dan support diperiksa, validator menghitung beban setiap truk. Untuk truk t, jumlah berat paket yang ditugaskan kepadanya harus memenuhi

$$\sum_{i \in P_t} \text{weight}_i \leq \text{max_capacity}_t. \tag{5}$$

Cuplikan 9 menunjukkan bahwa berat dijumlahkan dengan `Decimal`. Dengan cara ini, batas seperti $0.1 + 0.2 \leq 0.3$ tidak keliru akibat pembulatan bilangan *floating point*. Hasil `validate_state` berupa `ValidationResult` yang memuat status `valid`, daftar `errors`, dan total berat per truk. Jika assignment atau geometri sudah ambigu, pemeriksaan support dilewati karena grid tidak lagi dapat dipercaya, sementara kesalahan yang sudah ditemukan tetap dilaporkan.

Cuplikan 9. Penjumlahan berat dan pemeriksaan kapasitas

```
exact_weights = {truck.id: Decimal(0) for truck in problem.trucks}
for placement in loaded:
    assert placement.truck_id is not None
    exact_weights[placement.truck_id] += Decimal(
        str(packages[placement.package_id].weight)
    )
truck_weights = {
    truck_id: float(weight) for truck_id, weight in exact_weights.items()
}
for truck in problem.trucks:
    if exact_weights[truck.id] > Decimal(str(truck.max_capacity)):
        errors.append(
            f"Truck {truck.id} exceeds capacity: "
            f"{truck_weights[truck.id]} > {truck.max_capacity}"
        )
```

3.3.4 Objective Function

Setelah state dinyatakan sah, program menghitung nilainya melalui `ObjectiveMode` pada `app/optimization/objective.py`. **Mode utama**, `TOTAL_VALUE`, mengimplementasikan fungsi objektif pada Persamaan 2 di Subbab 1.6; agregasi seluruh armada merupakan perluasan atas rumusan konseptual satu truk:

$$f_{\text{total}}(s) = \sum_{i:\, \text{truck}_i \neq \text{None}} \text{value}_i. \tag{6}$$

Mode tambahan, `ETA_WEIGHTED_VALUE`, memberi bobot lebih besar pada paket yang ETA-nya lebih dekat. Ini ekstensi opsional, bukan objektif konseptual utama:

$$f_{\text{ETA}}(s) = \sum_{i:\, \text{truck}_i \neq \text{None}} \frac{\text{value}_i}{\text{eta}_i + 1}. \tag{7}$$

Kontribusi setiap paket termuat dihitung oleh `objective_contributions`, lalu dijumlahkan oleh `evaluate_objective` dengan `math.fsum`. Evaluator tidak memberi penalti atas pelanggaran constraint dan tidak memeriksa kelayakan state. Dalam HC dan SA, neighbor yang dinilai terlebih dahulu harus sah. Pada GA, objective mentah tetap tidak berpenalti, tetapi offspring tidak sah juga dievaluasi; fitness terpisah menerapkan penalti berdasarkan jumlah error validator.

3.4 Implementasi Algoritma *Local Search*

3.4.1 Inisialisasi State dan Pembangkitan Neighbor

Sebelum pencarian dimulai, `runner.py` membuat satu *random number generator* dari *seed* yang diterima. Generator yang sama diteruskan ke inisialisasi dan algoritma agar run dengan input serta seed yang sama dapat diulang. Hill climbing dan simulated annealing menerima satu initial state dari `generate_random_feasible_state`. Genetic algorithm memakai fungsi yang sama untuk membentuk anggota populasi awal.

Untuk memenuhi ketentuan state awal acak pada Subbab 1.4, inisialisasi memulai semua paket di luar truk, lalu mengacak urutan paket dan tujuan penempatannya. Untuk setiap truk yang dicoba, fungsi memilih orientasi dan posisi. Koordinat x dan y diambil dari rentang yang masih muat, sedangkan z diambil dari lantai atau permukaan atas paket lain. Kandidat baru hanya disimpan bila lolos `validate_state`. Paket dapat tetap di luar saat target luar terpilih atau ketika percobaan penempatan gagal. Cuplikan 10 menunjukkan titik awal konstruksi tersebut.

Cuplikan 10. Pembentukan initial state dalam `initializer.py`

```
state = State(
    placements=[PackagePlacement(package_id=p.id) for p in problem.packages]
)
indices = list(range(len(problem.packages)))
rng.shuffle(indices)
```

Setelah initial state tersedia, neighbor dibentuk melalui tiga operasi pada Subbab 1.4. `RELOCATE` memindahkan paket ke koordinat lain, mengeluarkannya dari truk, atau memuatnya kembali; implementasi multi-truk juga memungkinkan transfer antartruk. `ROTATE` mengubah orientasi paket termuat sambil mempertahankan titik posisinya. `SWAP` menukar tujuan truk dan posisi dua paket, termasuk kasus satu paket di luar, sementara masing-masing mempertahankan orientasinya. Peluang pemilihan awalnya berturut-turut 0,50, 0,25, dan 0,25. Setiap move bekerja pada salinan state agar state asal tidak berubah.

Proposal move belum tentu memenuhi constraint. Untuk hill climbing dan simulated annealing, `generate_random_neighbor` mengulang proposal sampai menemukan state sah atau mencapai batas percobaan, seperti pada Cuplikan 11. Varian HC selain stochastic dapat meminta beberapa neighbor berbeda melalui `sample_valid_neighbors`; stochastic dan simulated annealing meminta satu neighbor. Pada genetic algorithm, offspring yang tidak sah tetap berada di populasi dan dinilai dengan fitness berpenalti, tanpa repair.

Cuplikan 11. Penerimaan neighbor yang sah dalam `neighbors.py`

```
for _ in range(config.max_attempts):
    candidate = propose_random_move(problem, state, rng, config)
    if candidate is not None and validate_state(problem, candidate.state).valid:
        return candidate
return None
```

3.4.2 Hill Climbing

Keempat varian hill climbing diimplementasikan dalam kelas `HillClimbing` pada `app/optimization/algorithms/hill_climbing.py`. Steepest-ascent, sideways, dan random-

restart mengambil sampel neighbor sah melalui `sample_valid_neighbors`; stochastic meminta satu melalui `generate_random_neighbor`. Jumlah kandidat untuk tiga varian pertama diatur oleh `candidates_per_iteration` (default 50). Generator dibatasi percobaannya (default 30), sehingga neighbor dapat tidak tersedia. Perbedaan tiap varian terletak pada cara memilih kandidat dan melanjutkan pencarian. Cuplikan berikut merangkum bagian implementasi yang digunakan masing-masing varian, dengan pencatatan riwayat tidak ditampilkan.

Steepest-ascent hill climbing membandingkan skor state saat ini dengan seluruh neighbor sah yang berhasil masuk ke sampel iterasi. Secara teoretis, steepest-ascent memilih neighbor terbaik dari seluruh neighborhood; implementasi hanya dapat memilih yang terbaik dari sampel, yang jumlahnya dibatasi dan dapat kurang dari target bila proposal sah tidak cukup ditemukan. Jadi, `local_optimum` menandai tidak adanya peningkatan pada sampel, bukan bukti bahwa semua neighbor sudah diperiksa. Langkah yang lebih baik dipilih, dan jika beberapa kandidat berbagi skor tertinggi, satu dipilih secara acak seperti pada Cuplikan 12. Aturan ini mengarahkan pencarian ke kenaikan tertinggi yang teramati, tetapi dapat berhenti di puncak lokal atau sebelum menemukan peningkat di luar sampel.

Cuplikan 12. Inti pemilihan neighbor pada steepest-ascent

```
top_score = max(scores)
if top_score <= current_score:
    termination_reason = "local_optimum"
    break
pool = [
    (n, score)
    for n, score in zip(neighbors, scores)
    if score == top_score
]
selected, selected_score = rng.choice(pool)
current, current_score = selected.state, selected_score
```

Stochastic hill climbing meminta satu neighbor acak setiap iterasi. Metode ini hanya menerima skor yang meningkat ketat. Skor setara atau lebih rendah mempertahankan state. Jika generator tidak menemukan neighbor, percobaan juga dicatat tanpa perpindahan. Kedua kondisi tersebut tidak menghentikan pencarian. Iterasi berlanjut hingga batas API, yaitu 2.000. Satu sampel gagal tidak membuktikan optimum lokal. Cuplikan 13 merangkum kebijakan saat ini.

Cuplikan 13. Satu neighbor acak per iterasi pada stochastic HC

```
candidate = generate_random_neighbor(problem, current, rng, neighbor_config)
if candidate is not None:
    candidate_score = evaluate_objective(problem, candidate.state, objective_mode)
    if candidate_score > current_score:
        current, current_score = candidate.state, candidate_score
```

Hill climbing with sideways move memilih kandidat dengan skor tertinggi dalam sampel seperti steepest-ascent, termasuk kandidat yang skornya sama dengan state saat ini. Langkah setara memungkinkan pencarian berpindah di plateau, tempat kenaikan mungkin belum terlihat dari posisi sekarang. Batas `max_sideways` berlaku untuk langkah setara berturut-turut: setiap langkah tersebut menambah penghitung, dan peningkatan

skor mengatur ulang penghitung ke nol. Dengan batas default 100, pencarian dapat mengambil hingga 100 langkah setara berurutan; jika langkah setara berikutnya masih diperlukan, pencarian berhenti dengan alasan `max_sideways`. Cuplikan 14 memperlihatkan aturan tersebut. Kandidat yang menurunkan skor tetap ditolak, dan jumlah langkah sideways dicatat. Batas ini mengendalikan penelusuran plateau, tetapi tidak menjamin bahwa plateau telah ditelusuri seluruhnya.

Cuplikan 14. Penerimaan dan penghitungan sideways move

```
if top_score > current_score or (
    top_score == current_score and consecutive_sideways < max_sideways
):
    pool = [
        (n, score)
        for n, score in zip(neighbors, scores)
        if score == top_score
    ]
    selected, selected_score = rng.choice(pool)
    improved = selected_score > current_score
    current, current_score = selected.state, selected_score
    if improved:
        consecutive_sideways = 0
    else:
        consecutive_sideways += 1
        sideways_moves += 1
```

Random-restart hill climbing memakai steepest-ascent untuk setiap climb, dimulai dari state awal runner lalu dilanjutkan dengan state sah baru pada tiap restart. Dengan 10 restart, implementasi melakukan 11 climb. Setiap climb mendapat batas `max_iterations` sendiri, sehingga anggaran itu berlaku per climb, bukan dibagi di antara semua restart. Cuplikan 15 memperlihatkan pengulangan dan pemilihan hasil: **state akhir** berasal dari climb terakhir, sedangkan `best_state` dan skor terbaik berasal dari climb dengan hasil tertinggi di seluruh pengulangan. Riwayat menggabungkan iterasi dengan penanda restart dan skor terbaik kumulatif. Restart memberi kesempatan untuk memulai dari wilayah lain dan mengurangi risiko bergantung pada satu puncak lokal, dengan biaya climb tambahan. Karena jumlahnya tetap terbatas dan tiap climb hanya memeriksa sampel neighborhood, metode ini tidak memastikan pencarian menemukan optimum global.

Varian selain stochastic dapat berhenti saat tidak menemukan neighbor sah. Stochastic tetap melanjutkan sampai batas iterasi. Batas API ialah 2.000 iterasi; sampling varian lain meminta hingga 50 kandidat, dan random-restart dibatasi hingga 20 restart. State saat ini dan terbaik disimpan terpisah.

Cuplikan 15. Pengulangan steepest-ascent pada random restart

```
start = initial_state
for restart in range(config.max_restarts + 1):
    if restart > 0:
        start = generate_random_feasible_state(problem, rng)
    climb = self._climb(
        problem,
        start,
        variant=HillClimbingVariant.STEEPEST_ASCENT,
        objective_mode=objective_mode,
        rng=rng,
        config=config,
        include_state_history=include_state_history,
    )
    climbs.append(climb)
best = max(climbs, key=lambda climb: climb.best_score)
```

3.4.3 Simulated Annealing

Kelas `SimulatedAnnealing` memulai pencarian dari state sah yang sama jenisnya dengan hill climbing, tetapi hanya mengambil **satu neighbor acak** pada setiap iterasi. Move dengan $\Delta f \geq 0$ langsung diterima. Untuk $\Delta f < 0$, algoritma menggunakan peluang $\exp(\Delta f / T)$ sebagaimana dijelaskan pada Bab 2. Cuplikan 16 menunjukkan keputusan tersebut di dalam kode.

Cuplikan 16. Penerimaan move pada `simulated_annealing.py`

```
delta = candidate_score - current_score
acceptance_probability = (
    1.0 if delta >= 0 else math.exp(delta / temperature)
)
accepted = delta >= 0 or rng.random() < acceptance_probability
if accepted:
    current = candidate.state.model_copy(deep=True)
    current_score = candidate_score
```

Temperatur awal dapat diberikan sebagai parameter. Jika kosong, nilainya diambil dari rata-rata value paket dengan batas bawah 1. Setelah satu iterasi selesai, temperatur dikalikan `alpha`, yang bernilai antara nol dan satu. Pencarian berhenti saat temperatur berada di bawah `min_temperature`, batas iterasi tercapai, atau neighbor sah tidak ditemukan. `best_state` disimpan terpisah dari state saat ini, sebab simulated annealing dapat menerima move yang menurunkan skor. Riwayat iterasi mencatat temperatur, peluang penerimaan, keputusan move, dan skor. Parameter `stuck_patience` hanya menandai dan menghitung kejadian ketika skor terbaik tidak membaik cukup lama; parameter ini tidak menghentikan pencarian.

3.4.4 Genetic Algorithm

Kelas `GeneticAlgorithm` memulai dari populasi feasible. Objective tetap dihitung tanpa penalti dan digunakan untuk skor hasil serta arsip solusi feasible. Seleksi memakai fitness terpisah: untuk nilai objective mentah V, batas atas U, dan jumlah error validator C, tetapkan $q = \text{clamp}(V/U, 0, 1)$ bila $U > 0$, selain itu $q = 0$. Fitness ialah $1 + q$ bila $C = 0$, dan $q/(1 + C)$ bila $C > 0$. Dengan demikian, penalti hanya memengaruhi

seleksi; C adalah hitungan diagnostik, bukan ukuran lengkap tingkat keparahan fisik, dan pemeriksaan dukungan dapat dilewati bila geometri ambigu.

Reproduksi memakai roulette berbobot fitness, single-point crossover dalam urutan paket kanonis, dan mutasi paling banyak satu gen melalui `propose_relocate`. Populasi generasi berikutnya dibentuk seluruhnya dari offspring, tanpa elitisme. Offspring tidak feasible tidak diperbaiki dan tetap dalam populasi. Arsip eksternal menyimpan solusi feasible terbaik, tidak ikut reproduksi; bila populasi akhir tidak memiliki solusi feasible, state akhir memakai arsip dan metadata menandai fallback. State awal, akhir, terbaik, dan snapshot riwayat tetap feasible serta skornya objective mentah.

Cuplikan berikut memperlihatkan pembentukan populasi baru:

Cuplikan 17. Pembentukan offspring GA

```
next_population = []
for _ in range(config.population_size):
    a = roulette_select(population, fitnesses, rng)
    b = roulette_select(population, fitnesses, rng)
    child = (
        single_point_crossover(problem, a, b, rng)
        if rng.random() < config.crossover_probability
        else rng.choice((a, b)).model_copy(deep=True)
    )
    if rng.random() < config.mutation_probability:
        proposal = propose_relocate(problem, child, rng)
        if proposal is not None:
            child = proposal.state
    next_population.append(child)
```

Default GA adalah populasi 50, 100 generasi, crossover 1,0, mutasi 0,2, dan elite 0; konfigurasi API membatasi populasi hingga 100 dan generasi hingga 200. Riwayat dimulai pada generasi 0 dan mencatat maksimum fitness di `metadata.max_fitness` serta rata-rata fitness di `mean_score`; keduanya bukan objective mentah yang ditampilkan sebagai skor current/best.

4 Eksperimen dan Analisis

4.1 Rancangan Eksperimen

Bagian ini menyajikan 15 pengukuran manual melalui UI (12 run Hill Climbing/HC dan tiga run Simulated Annealing/SA) serta tiga run GA yang dieksekusi secara berurutan melalui backend HTTP. Dengan demikian terdapat 18 catatan eksekusi pada satu dataset. Run GA merupakan pengujian programatik, bukan run manual melalui UI.

4.1.1 Data Uji dan Lingkungan Eksperimen

Objektif utama adalah **`TOTAL_VALUE`**, yaitu jumlah nilai paket termuat tanpa pembobotan ETA. Dataset pengujian terdiri atas satu truk `truck-a` berukuran $10 \times 8 \times 6$ dan berkapasitas 100, serta sembilan paket pada Tabel 2 dengan atribut asli. Dataset yang sama digunakan untuk membandingkan hasil antaralgoritma dan seed. GA memakai satu konfigurasi tetap: populasi 50, 100 generasi, crossover 1,0, mutasi 0,2, elitisme 0, dan arsip solusi feasible eksternal.

ID	Dimensi (w, l, h)	Nilai	Berat	Rapuh	ETA
P01	$(4, 3, 2)$	40	20	Tidak	2
P02	$(3, 3, 2)$	30	15	Tidak	1
P03	$(2, 2, 1)$	25	5	Ya	0
P04	$(2, 4, 3)$	35	18	Tidak	3
P05	$(3, 2, 2)$	20	10	Tidak	4
P06	$(1, 1, 1)$	8	2	Ya	1
P07	$(5, 4, 3)$	50	30	Tidak	2
P08	$(6, 6, 6)$	60	40	Tidak	5
P09	$(2, 2, 2)$	12	6	Tidak	0

Tabel 2. Data paket pada dataset pengujian.

Jumlah nilai seluruh paket adalah 280. Dengan batas kapasitas 100 kg dan aturan penempatan yang digunakan, nilai optimum global dataset ini adalah 208; pembuktiannya diberikan pada Subbab 4.3.1. Pengujian difokuskan pada satu truk dengan batas kapasitas 100 kg.

4.1.2 Konfigurasi Parameter dan Pengulangan Eksperimen

Setiap metode memiliki **tiga pengulangan dengan seed 42, 123, dan 2026**. Rancangan mencakup empat varian HC masing-masing tiga run, tiga run SA, serta satu konfigurasi GA dengan tiga run; keseluruhannya **18 catatan eksekusi pada satu dataset**. Konfigurasi parameter setiap metode dirangkum pada Tabel 3.

Metrik pengujian meliputi skor objektif awal, akhir, dan terbaik, runtime, jumlah iterasi aktual, serta berat dan jumlah paket termuat pada state akhir. Skor terbaik dibedakan dari skor akhir karena solusi terakhir tidak selalu menjadi solusi terbaik sepanjang pencarian. Runtime GA mencakup inisialisasi dan pencarian, tidak termasuk serialisasi, validasi akhir, penyimpanan, komunikasi HTTP, atau rendering. Pengukuran manual HC/SA tidak merupakan benchmark terkontrol terhadap GA. Runtime GA dijalankan berurutan pada Docker, CPython 3.12.13/Linux, Intel Core i7-13620H.

Metode	Konfigurasi	Run
HC steepest-ascent	Maksimum 2.000 iterasi, 50 kandidat per iterasi; tanpa sideways atau restart.	3
HC stochastic	Maksimum 2.000 iterasi; satu neighbor acak per iterasi, diterima jika skornya meningkat.	3
HC sideways	Maksimum 2.000 iterasi, 50 kandidat per iterasi, maksimum 100 sideways berturut-turut.	3
HC random-restart	Maksimum 10 restart; 11 climb, masing-masing maksimum 2.000 iterasi dan 50 kandidat per iterasi.	3
SA	$T_0 = 30$, $\alpha = 0{,}995$, $T_{\min} = 0{,}001$, maksimum 5.000 iterasi, ambang stagnasi 50 iterasi.	3
GA	Populasi 50; 100 generasi; crossover 1,0; mutasi 0,2; elitisme 0.	3

Tabel 3. Kontrol dan jumlah run per kelompok eksperimen.

4.2 Hasil Eksperimen dan Visualisasi Program

Pengujian mencakup empat varian HC, SA, dan GA dengan objektif Total value. Hasil setiap run meliputi skor awal, akhir, dan terbaik. Visualisasi HC/SA menunjukkan susunan paket pada state akhir beserta jumlah paket dan berat muatannya. Hasil GA diperoleh melalui backend dan belum dilengkapi tangkapan layar.

4.2.1 Hill Climbing

Metode	Seed	Awal	Akhir	Terbaik	ms	Iterasi	kg
Steepest-ascent	42	188	208	208	403	2	100
Steepest-ascent	123	68	170	170	632	5	76
Steepest-ascent	2026	93	208	208	488	4	100
Stochastic	42	188	208	208	932	2000	100
Stochastic	123	68	170	170	1148	2000	76
Stochastic	2026	93	185	185	1150	2000	88
Sideways	42	188	208	208	16530	102	100
Sideways	123	68	208	208	16996	108	100
Sideways	2026	93	208	208	16100	104	100
Random-restart	42	188	190	208	5064	49	96
Random-restart	123	68	190	208	4135	42	91
Random-restart	2026	93	208	208	4794	42	100

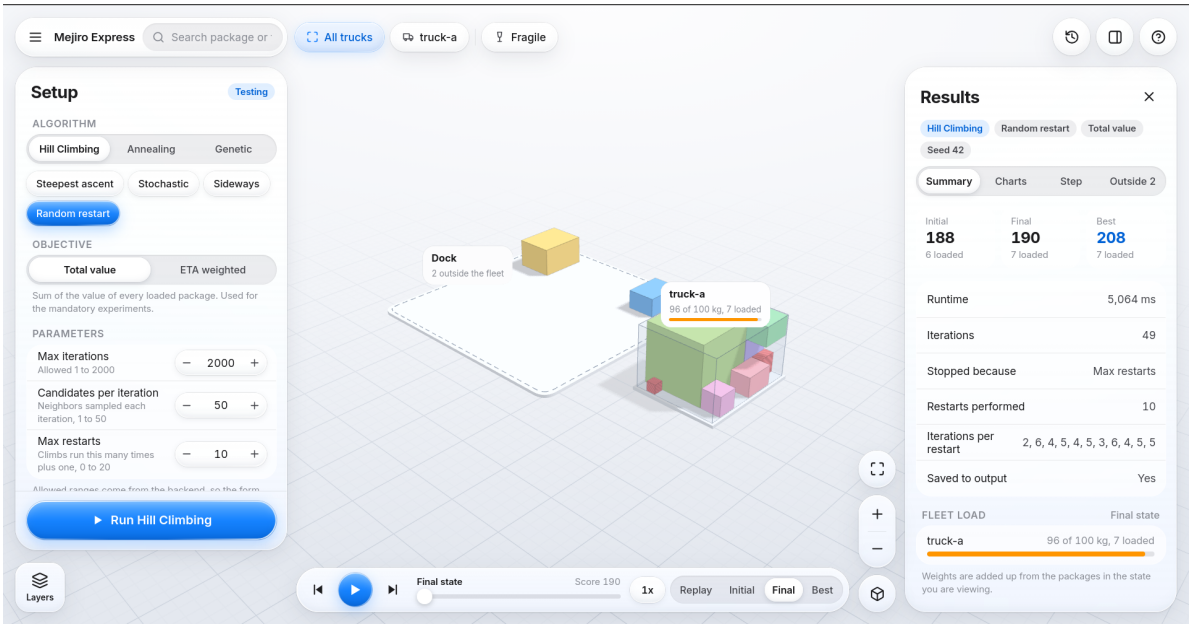
Tabel 4. Hasil pengukuran manual 12 run HC. Kolom kg adalah berat muatan akhir; seluruh run berakhir dengan tujuh paket termuat.

Steepest-ascent berhenti dengan alasan *Local optimum* pada seluruh seed dan menghasilkan skor akhir 208, 170, dan 208. Status tersebut berarti tidak ditemukan peningkatan pada kandidat yang diperiksa saat penghentian. Stochastic menghasilkan skor akhir sekaligus skor terbaik 208, 170, dan 185, dengan berat akhir 100, 76, dan 88 kg. Ketiga run stochastic menjalankan 2.000 iterasi dan berhenti karena *Max iterations*. Kandidat yang tidak meningkatkan skor ditolak tanpa menghentikan pencarian.

Sideways mencapai skor akhir 208 pada ketiga seed dan berhenti karena *Max sideways*. Jumlah iterasi aktual 102, 108, dan 104 lebih kecil daripada batas 2.000 iterasi. Ringkasan

UI mencatat jumlah sideways total 100, 101, dan 100; angka total ini perlu dibedakan dari batas 100 sideways berturut-turut pada konfigurasi.

Random-restart menjalankan 10 restart dan berhenti karena *Max restarts*. Skor terbaik selalu 208, tetapi skor akhir pada seed 42 dan 123 adalah 190. Selisih tersebut menunjukkan perlunya membedakan solusi terakhir dengan solusi terbaik sepanjang seluruh climb. Pada dua run tersebut berat muatan akhir masing-masing 96 kg dan 91 kg, meskipun nilai akhir sama. Ketiga seed tetap memuat tujuh paket pada state akhir.



Gambar 7. Random-restart seed 42: skor akhir 190, skor terbaik 208, 49 iterasi, dan 10 restart. Pilihan Final menampilkan muatan akhir 96 kg.

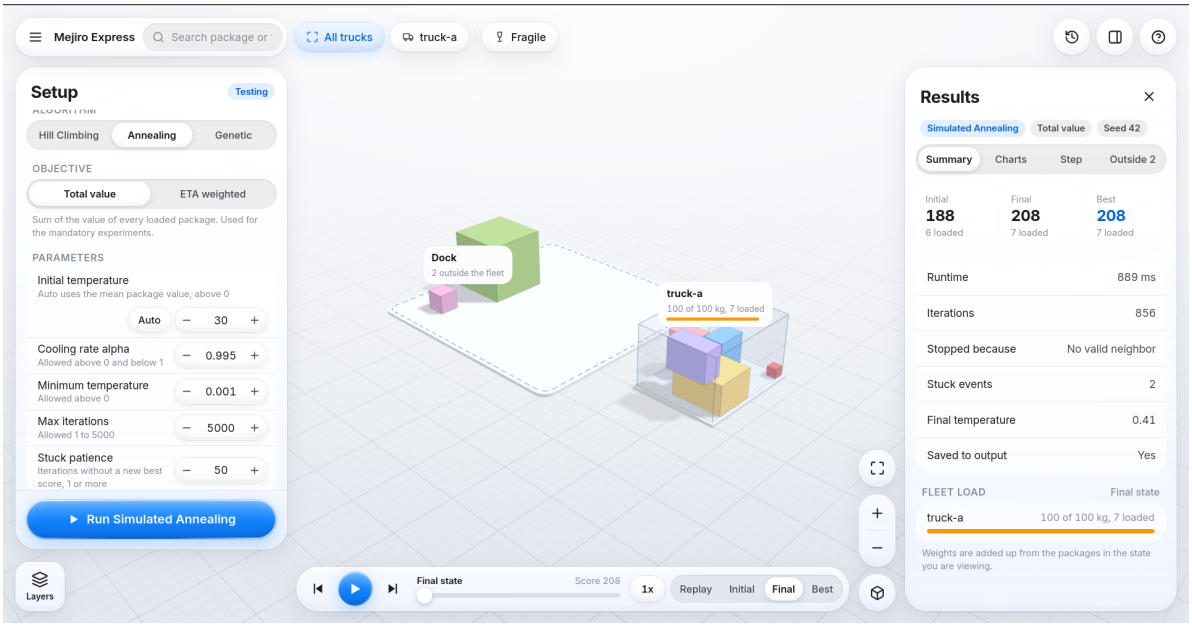
4.2.2 Simulated Annealing

Metode	Seed	Awal	Akhir	Terbaik	ms	Iterasi	kg
SA	42	188	208	208	889	856	100
SA	123	68	208	208	2014	2057	100
SA	2026	93	208	208	1998	2057	100

Tabel 5. Hasil pengukuran manual tiga run SA. Seluruh state akhir memuat tujuh paket.

SA mencapai skor akhir dan terbaik 208 pada seluruh seed. Seed 42 berhenti karena *No valid neighbor* setelah 856 iterasi, dengan suhu akhir yang ditampilkan 0,41. Seed 123 dan 2026 berhenti karena *Temperature minimum* setelah 2.057 iterasi, dengan suhu akhir ditampilkan 0,001. Jumlah *stuck events* masing-masing adalah 2, 2, dan 3; metrik ini bukan jumlah iterasi stagnasi berturut-turut. Penghentian aktual pada ketiga run terjadi sebelum batas 5.000 iterasi.

Konfigurasi SA menggunakan $T_0 = 30$, $\alpha = 0{,}995$, $T_{\min} = 0{,}001$, dan ambang stagnasi 50.



Gambar 8. SA seed 42: pengaturan parameter, state akhir, skor awal/akhir/terbaik, runtime, dan alasan penghentian.

4.2.3 Genetic Algorithm

Tiga run GA menggunakan satu konfigurasi tetap (populasi 50, 100 generasi) dan seed 42, 123, serta 2026. Tabel melaporkan objective mentah (total value), runtime backend, jumlah paket, dan berat state akhir. Seluruh state awal/akhir/terbaik lolos validasi; seluruh run mencapai generation limit pada tepat $I = G = 100$ generasi dan tidak memakai fallback arsip sebagai state akhir.

Seed	Awal	Akhir	Terbaik	ms	Paket	kg	Berkas
42	200	208	208	875	7	100	`ga-seed-42.json`
123	208	162	208	787	6	74	`ga-seed-123.json`
2026	200	208	208	898	7	100	`ga-seed-2026.json`

Tabel 6. Hasil GA programatik untuk satu konfigurasi tetap (populasi 50, 100 generasi, crossover 1,0, mutasi 0,2, elitisme 0), tiga seed.

Rata-rata skor akhir adalah 192,67 dengan simpangan baku populasi 21,68; rata-rata skor terbaik 208,00 dan runtime backend rata-rata 853 ms. Skor akhir seed 123 sebesar 162 berada di bawah skor terbaik 208 karena reproduksi tidak memakai elitisme. Arsip solusi terbaik disimpan terpisah dan tidak memaksa state akhir sama dengan arsip.

4.3 Analisis Eksperimen

Seluruh ringkasan dihitung dari tiga seed per metode. Simpangan baku yang digunakan adalah simpangan baku populasi deskriptif, $\sigma = \sqrt{\sum_{i=1}^{3}(x_i - \bar{x})^2/3}$. Jumlah pengulangan ini terbatas dan tidak digunakan untuk menyatakan signifikansi statistik.

Metode	$\bar{f}_{akhir}$	σ_{akhir}	$\bar{f}_{terbaik}$	$\bar{t}$ (s)	$\bar{I}$
Steepest-ascent	195.33	17.91	195.33	0.508	3.67
Stochastic	187.67	15.63	187.67	1.077	2000.00
Sideways	208.00	0.00	208.00	16.542	104.67
Random-restart	196.00	8.49	208.00	4.664	44.33
SA	208.00	0.00	208.00	1.634	1656.67
GA	192.67	21.68	208.00	0.853	100.00

Tabel 7. Ringkasan deskriptif tiga seed per metode: rata-rata skor akhir, simpangan baku populasi skor akhir, rata-rata skor terbaik, runtime, dan iterasi aktual. Runtime GA berasal dari kondisi eksekusi berbeda dari HC/SA.

4.3.1 Kedekatan terhadap Optimum Global

Nilai optimum global dapat ditentukan untuk dataset sembilan paket ini. Jika batas ruang, support, dan fragile diabaikan sementara, setiap solusi tetap harus memenuhi batas berat. Dengan $x_i \in \{0, 1\}$ sebagai penanda paket termuat, diperoleh batas atas

$$U = \max \left\{ \sum_{i=1}^{9} v_i x_i \;\middle|\; \sum_{i=1}^{9} w_i x_i \leq 100 \right\}.$$

Pemeriksaan seluruh $2^9 = 512$ subset menghasilkan $U = 208$. Subset yang mencapai nilai tersebut adalah P01, P02, P03, P04, P05, P06, dan P07 dengan berat total 100 kg. Karena pemeriksaan ini melonggarkan kendala penempatan, nilai solusi sah pada persoalan asli tidak dapat melebihi 208.

Batas tersebut dapat dicapai dengan susunan pada Tabel 8. Semua paket memakai orientasi dimensi asli dan berada di lantai truk; P08 dan P09 berada di luar. Susunan ini berada dalam ukuran truk $10 \times 8 \times 6$, tanpa overlap, dan tidak menempatkan paket di atas paket fragile. Pemeriksaan dengan validator menghasilkan state sah tanpa pelanggaran. Adanya solusi sah bernilai 208 dan batas atas 208 membuktikan bahwa $f^* = 208$ untuk dataset dan aturan kendala ini. Susunan tersebut menjadi sertifikat kelayakan untuk pembuktian, bukan rekonstruksi posisi paket dari run yang tercatat.

Paket	Dimensi (w, l, h)	Posisi (x, y, z)	Nilai	Berat (kg)
P01	$(4, 3, 2)$	$(5, 0, 0)$	40	20
P02	$(3, 3, 2)$	$(5, 3, 0)$	30	15
P03	$(2, 2, 1)$	$(2, 6, 0)$	25	5
P04	$(2, 4, 3)$	$(0, 4, 0)$	35	18
P05	$(3, 2, 2)$	$(2, 4, 0)$	20	10
P06	$(1, 1, 1)$	$(4, 6, 0)$	8	2
P07	$(5, 4, 3)$	$(0, 0, 0)$	50	30
Total			208	100

Tabel 8. Satu susunan sah yang mencapai batas atas nilai 208. Posisi menyatakan sudut bawah paket menurut sumbu lebar, panjang, dan tinggi.

Kedekatan hasil pencarian diukur melalui gap relatif $g(f) = 100(f^* - f)/f^*$. Skor 208 memiliki gap 0%, skor 190 memiliki gap 8,65%, skor 185 memiliki gap 11,06%, dan skor 170 memiliki gap 18,27%. Tabel 9 memisahkan gap skor akhir dari gap skor terbaik.

Steepest-ascent berhenti ketika sampel kandidat tidak menyediakan peningkatan. Pada seed 123, hasilnya 170, meskipun solusi global 208 tersedia. Penghentian berlabel *Local*

Metode	$\bar{g}_{akhir}$ (%)	$\bar{g}_{terbaik}$ (%)	Akhir optimum	Terbaik optimum
Steepest-ascent	6,09	6,09	2/3	2/3
Stochastic	9,78	9,78	1/3	1/3
Sideways	0,00	0,00	3/3	3/3
Random-restart	5,77	0,00	1/3	3/3
SA	0,00	0,00	3/3	3/3
GA	7,37	0,00	2/3	3/3

Tabel 9. Gap rata-rata terhadap optimum 208 serta jumlah run yang mencapai optimum dari tiga seed.

optimum berlaku terhadap kandidat yang diperiksa; sampling tersebut tidak mencakup seluruh neighbor yang mungkin. Stochastic tetap mencoba satu neighbor acak sampai batas 2.000 iterasi, tetapi hanya menerima peningkatan ketat. Metode ini mencapai optimum pada seed 42, sedangkan seed 123 dan 2026 berakhir di 170 dan 185. Gap yang tersisa menunjukkan bahwa penambahan percobaan neighbor dalam anggaran ini belum cukup untuk memperoleh optimum pada kedua seed tersebut. Stochastic menolak langkah setara dan menurun, sehingga pilihan perpindahannya terbatas. Ringkasan akhir belum membuktikan bahwa state terakhir adalah optimum lokal untuk semua neighbor.

Sideways menerima langkah bernilai sama sehingga dapat mengubah susunan paket tanpa langsung meningkatkan objektif. Kemampuan tersebut menyediakan jalur untuk melanjutkan pencarian pada plateau dan sesuai dengan hasil akhirnya yang mencapai 208 pada semua seed. Setelah nilai optimum tercapai, langkah setara masih dapat berlanjut sampai batas sideways tercapai. Tanpa riwayat skor per iterasi, bagian lintasan yang melewati plateau tidak dapat ditentukan dari ringkasan akhir saja.

Random-restart memulai climb baru dari state acak sehingga pencarian mencoba lebih dari satu daerah solusi. Skor terbaik 208 pada seluruh seed menunjukkan bahwa mekanisme ini berhasil menemukan optimum dalam setiap run. Pada seed 42 dan 123, climb terakhir berakhir di 190 sehingga skor akhirnya lebih rendah daripada optimum yang sudah ditemukan. Solusi terbaik sepanjang run perlu digunakan ketika memilih susunan untuk diterapkan.

SA dapat menerima langkah yang menurunkan skor dengan probabilitas yang bergantung pada suhu. Mekanisme ini memungkinkan pencarian meninggalkan daerah yang sulit diperbaiki hanya dengan langkah meningkat. Ketiga run SA mencapai 208, termasuk seed 123 yang menghasilkan 170 pada steepest-ascent dan stochastic. Hasil ini sesuai dengan kemampuan eksplorasi SA, tetapi ringkasan akhir tidak menunjukkan langkah menurun mana yang berperan dalam mencapai optimum.

4.3.2 Perbandingan Hasil Pencarian Antaralgoritma

Untuk HC/SA, berdasarkan rata-rata skor akhir, sideways dan SA berada pada urutan tertinggi dengan 208, diikuti random-restart 196,00, steepest-ascent 195,33, dan stochastic 187,67. Selisih random-restart terhadap steepest-ascent hanya 0,67 poin pada rata-rata tiga seed, tetapi keduanya berbeda dalam penyimpanan hasil terbaik: random-restart menemukan 208 pada semua run, sedangkan steepest-ascent hanya pada dua run.

Pada seed 42, semua metode mencapai skor akhir 208 kecuali random-restart, yang berakhir di 190 setelah sempat mencapai 208. Pada seed 123, sideways dan SA meningkatkan skor awal 68 menjadi 208. Steepest-ascent dan stochastic berakhir di 170, sedangkan

random-restart menemukan skor terbaik 208. Pada seed 2026, skor terbaik stochastic adalah 185; keempat metode lainnya mencapai 208.

Seluruh state akhir HC/SA memuat tujuh paket; GA memuat enam hingga tujuh paket. Berat akhir HC/SA berkisar 76 sampai 100 kg, sedangkan nilainya berkisar 170 sampai 208. Karena objektif memaksimalkan total nilai, jumlah paket yang sama tidak menjamin kualitas solusi yang sama. Dua run random-restart juga memiliki nilai akhir 190 dengan berat berbeda, yaitu 96 dan 91 kg. Pemilihan komposisi paket dan susunan yang memungkinkan paket bernilai tinggi masuk lebih menentukan daripada jumlah paket saja.

4.3.3 Perbandingan Durasi Pencarian

Durasi HC/SA manual dan GA programatik berasal dari kondisi eksekusi berbeda sehingga tidak membentuk benchmark terkontrol. Perbandingan berikut dibatasi pada hasil HC/SA manual. Rata-rata runtime steepest-ascent adalah 0,508 detik, stochastic 1,077 detik, SA 1,634 detik, random-restart 4,664 detik, dan sideways 16,542 detik. Steepest-ascent paling cepat pada rata-rata pengukuran ini, tetapi tidak mencapai optimum pada seed 123. Sideways dan SA sama-sama mencapai skor akhir optimum pada seluruh seed; untuk hasil tersebut, SA membutuhkan sekitar sepersepuluh runtime sideways.

Steepest-ascent selesai dalam 2 sampai 5 iterasi karena berhenti ketika tidak menemukan peningkatan pada sampel kandidat. Stochastic menjalankan seluruh 2.000 iterasi dengan runtime 0,932 sampai 1,150 detik. Percobaan yang ditolak tetap dihitung sebagai iterasi, sehingga angka tersebut tidak sama dengan jumlah perpindahan state. Sideways menjalankan 102 sampai 108 iterasi dan terus memeriksa kandidat pada langkah bernilai sama, sehingga membutuhkan waktu lebih lama. Random-restart mengulang proses climb sampai 10 restart; biaya pengulangan itu menaikkan runtime dibanding satu climb, sekaligus menghasilkan skor terbaik optimum pada seluruh seed.

SA menjalankan 856 sampai 2.057 iterasi dengan runtime 0,889 sampai 2,014 detik. Biaya satu iterasi tidak setara antaralgoritma: steepest-ascent, sideways, dan random-restart memeriksa hingga 50 kandidat per iterasi, sedangkan stochastic dan SA meminta satu neighbor. Karena itu, jumlah iterasi SA yang lebih besar tetap dapat menghasilkan runtime lebih rendah daripada sideways. Pada seed 42, SA berhenti karena tidak memperoleh neighbor sah; dua seed lainnya berhenti saat suhu mencapai batas minimum. Perbedaan penghentian ini menjelaskan sebagian variasi jumlah iterasi dan durasinya.

Jika tujuan pemakaian adalah memperoleh skor akhir optimum pada ketiga seed ini, SA memberikan kombinasi kualitas dan durasi yang lebih baik daripada sideways. Jika waktu pencarian lebih diutamakan, steepest-ascent menghasilkan solusi dalam kurang dari satu detik pada ketiga run, dengan risiko skor di bawah optimum. Kesimpulan ini berlaku pada dataset dan konfigurasi yang diuji; runtime yang dibandingkan mencakup inisialisasi dan pencarian.

4.3.4 Konsistensi Hasil Akhir

Skor awal HC/SA untuk seed 42, 123, dan 2026 adalah 188, 68, dan 93 pada seluruh metode tersebut. Dengan demikian, setiap seed menyediakan titik pembandingan skor awal yang sama. Kesamaan skor tersebut tidak membuktikan bahwa koordinat semua paket identik.

Sideways dan SA menghasilkan skor akhir 208 pada tiga dari tiga run dengan simpangan baku nol. Steepest-ascent menghasilkan 208, 170, dan 208 dengan simpangan baku

17,91, sedangkan stochastic menghasilkan 208, 170, dan 185 dengan simpangan baku 15,63. Stochastic memiliki sebaran skor lebih kecil, tetapi rata-ratanya 187,67 berada di bawah steepest-ascent yang mencapai 195,33. Frekuensi pencapaian optimum stochastic adalah satu dari tiga run, sedangkan steepest-ascent dua dari tiga run. Oleh sebab itu, penilaian konsistensi perlu menyertakan rata-rata dan frekuensi pencapaian optimum, selain simpangan baku.

Random-restart menghasilkan skor akhir 190, 190, dan 208 dengan simpangan baku 8,49. Skor terbaiknya selalu 208 sehingga simpangan baku skor terbaik adalah nol. Metode ini konsisten menemukan optimum, sementara solusi pada akhir climb terakhir masih bervariasi. Perbedaan tersebut menjelaskan mengapa evaluasi yang hanya memakai skor akhir akan melewatkan keberhasilan pencarian sebelumnya.

Konsistensi skor juga perlu dibedakan dari konsistensi runtime. SA memiliki skor akhir tetap, tetapi runtime bervariasi dari 0,889 sampai 2,014 detik akibat perbedaan lama pencarian. Sideways memiliki rentang runtime 16,100 sampai 16,996 detik dengan skor akhir yang sama. Tiga seed ini menunjukkan perilaku pada sampel pengujian dan belum membuktikan tingkat keberhasilan untuk seed lain atau dataset yang lebih besar.

4.3.5 Analisis Genetic Algorithm pada Konfigurasi Tetap

Pengujian GA menggunakan populasi 50, 100 generasi, probabilitas crossover 1,0, probabilitas mutasi 0,2, dan tanpa elitisme. Tiga run memakai seed 42, 123, dan 2026. Analisis berikut menilai kualitas hasil dan variasi antarseed pada konfigurasi tersebut.

Skor terbaik ketiga run adalah 208, sama dengan optimum global yang dibuktikan pada Subbab 4.3.1. Riwayat pencarian menunjukkan bahwa seed 42 pertama kali mencapai skor terbaik 208 pada generasi 36, sedangkan seed 2026 pada generasi 52. Pada seed 123, skor 208 sudah tersedia pada populasi awal, yaitu generasi 0. Dengan demikian, dua run memperbaiki skor awal 200 menjadi 208, sementara satu run sudah memiliki solusi optimum sebelum reproduksi dimulai. Untuk ketiga seed ini, anggaran 100 generasi mencakup titik pertama ditemukannya optimum. Hasil tersebut tidak menjamin anggaran yang sama akan cukup untuk semua seed atau dataset lain.

Skor akhir GA adalah 208, 162, dan 208. Pada seed 123, populasi akhir menghasilkan solusi dengan nilai 162, enam paket termuat, dan berat 74 kg. Skor ini memiliki gap 22,12% terhadap optimum. Reproduksi tanpa elitisme memungkinkan solusi terbaik hilang dari populasi berikutnya; arsip eksternal tetap menyimpan solusi sah terbaik bernilai 208. Karena itu, perbedaan skor akhir dan terbaik perlu diperhatikan ketika memilih keluaran GA untuk digunakan. Dua seed lainnya berakhir dengan tujuh paket, berat 100 kg, dan gap nol.

Rata-rata skor akhir adalah 192,67, simpangan baku populasi 21,68, dan gap rata-rata 7,37%. Berdasarkan skor akhir, GA mencapai optimum pada dua dari tiga run. Berdasarkan skor terbaik, GA mencapai optimum pada tiga dari tiga run dengan simpangan baku nol. Variasi skor akhir tersebut menunjukkan bahwa keberhasilan menemukan optimum tidak selalu diikuti keberhasilan mempertahankannya dalam populasi terakhir.

Dibandingkan metode lain, rata-rata skor akhir GA berada di bawah sideways dan SA (208), random-restart (196,00), serta steepest-ascent (195,33), tetapi di atas stochastic (187,67). Jika solusi terbaik sepanjang pencarian yang dipilih, GA setara dengan sideways, random-restart, dan SA karena semuanya mencapai 208 pada setiap seed. Perbandingan peningkatan skor perlu mempertimbangkan inisialisasi: GA dimulai dari populasi dengan skor awal 200, 208, dan 200, sedangkan HC/SA dimulai dari satu state dengan skor awal 188, 68, dan 93.

Runtime GA adalah 875, 787, dan 898 ms, dengan rata-rata 853 ms atau 0,853 detik. Seluruh run menyelesaikan 100 generasi dan berhenti karena batas generasi. Durasi berkisar 0,787 sampai 0,898 detik, meskipun skor akhirnya bervariasi. Kondisi eksekusi GA programatik berbeda dari HC/SA manual sehingga angka tersebut menggambarkan durasi ketiga run GA, tanpa menjadi dasar klaim bahwa GA lebih cepat daripada metode lain.

Jumlah populasi dan generasi tetap pada ketiga run, sehingga perbedaan hasil berkaitan dengan seed pada konfigurasi yang sama. Data ini belum mengukur pengaruh perubahan kedua parameter tersebut. Oleh sebab itu, hasil pengujian hanya mendukung kesimpulan bahwa populasi 50 dan anggaran 100 generasi mampu menghasilkan solusi terbaik optimum pada ketiga seed yang diuji; pengujian ini tidak menetapkan ukuran populasi atau jumlah generasi yang paling efisien.

5 Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

6 Pembagian Tugas Tiap Anggota Kelompok

Nama Lengkap	NIM	Tugas/Kontribusi
Samuelson Dharmawan Tanuraharja	13524001	
Nicholas Wise Saragih Sumbayak	13524037	
Neswa Eka Anggara	13524136	

Tabel 10. Pembagian tugas anggota kelompok

Daftar Pustaka

[1] Tim Asisten Laboratorium Inteligensi Artifisial, "Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027," ver. 1.4, Institut Teknologi Bandung, Bandung, Indonesia, Sep. 2026.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

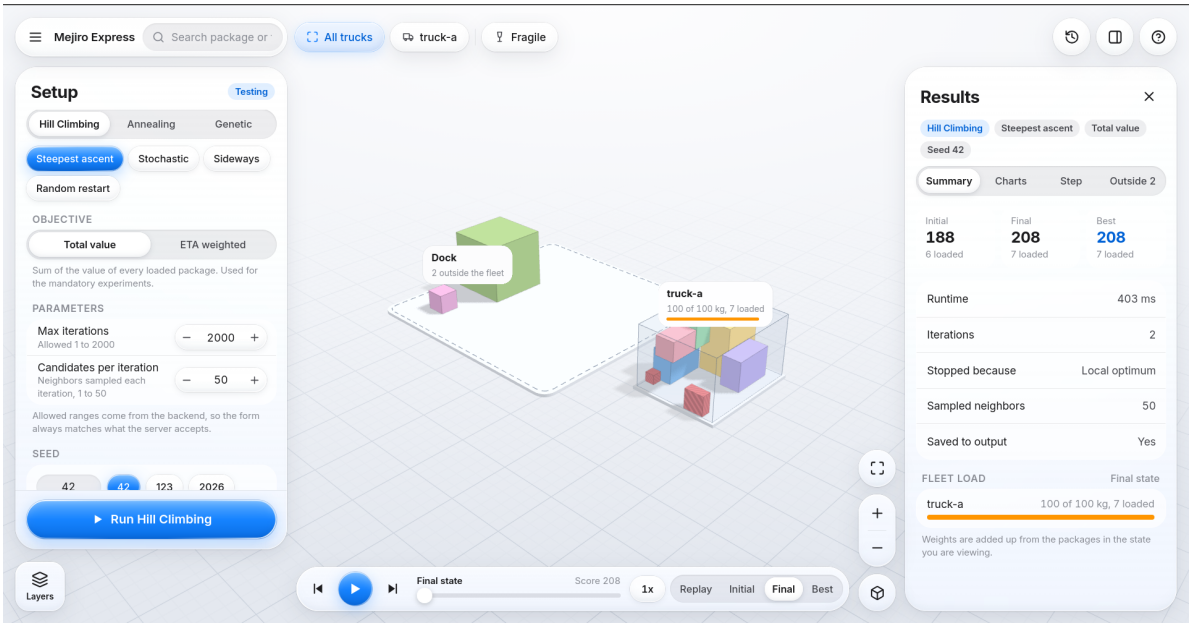
[3] Tim Pengajar IF3170 Inteligensi Artifisial, "Beyond Classical Search," lecture slides, Institut Teknologi Bandung, Bandung, Indonesia, 2026.

A Form Penggunaan Generative AI

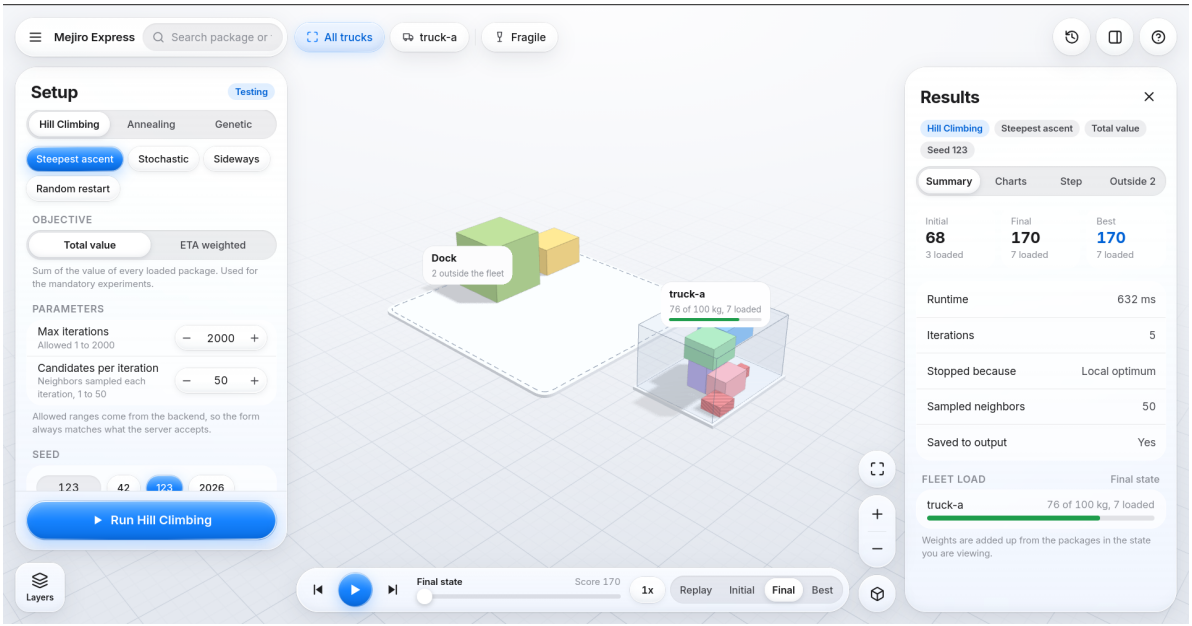
B Lampiran Lainnya (Bila Ada)

C Bukti Pengujian Manual

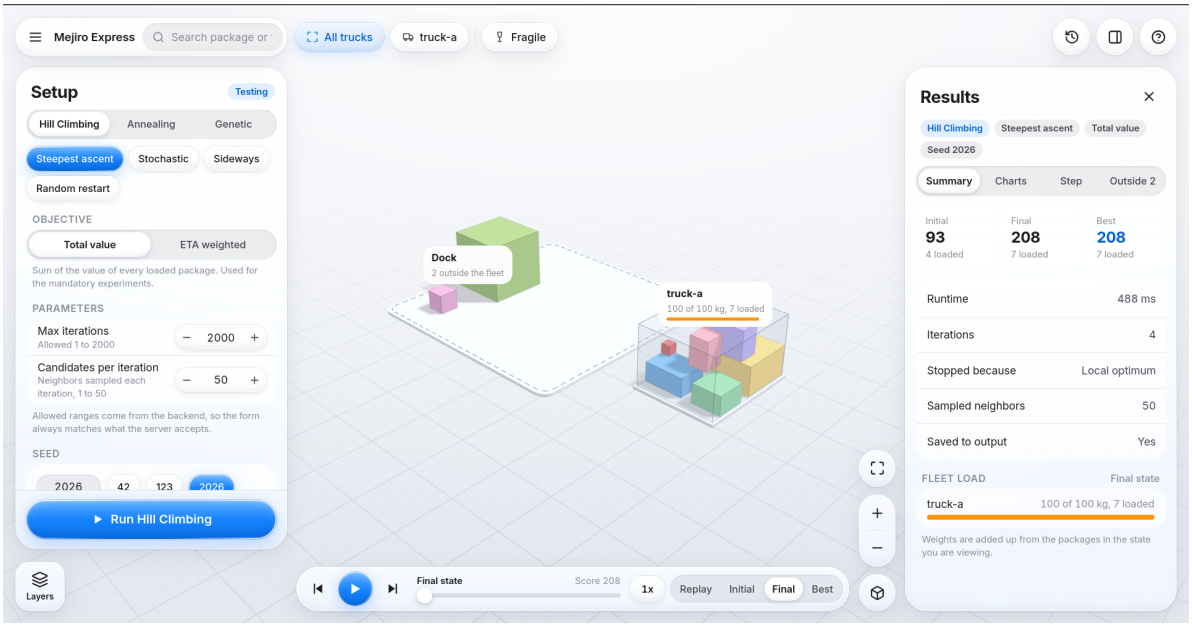
Tangkapan layar berikut memperlihatkan state akhir, skor objektif, runtime, dan jumlah iterasi setiap run, melengkapi Gambar 7 dan Gambar 8.



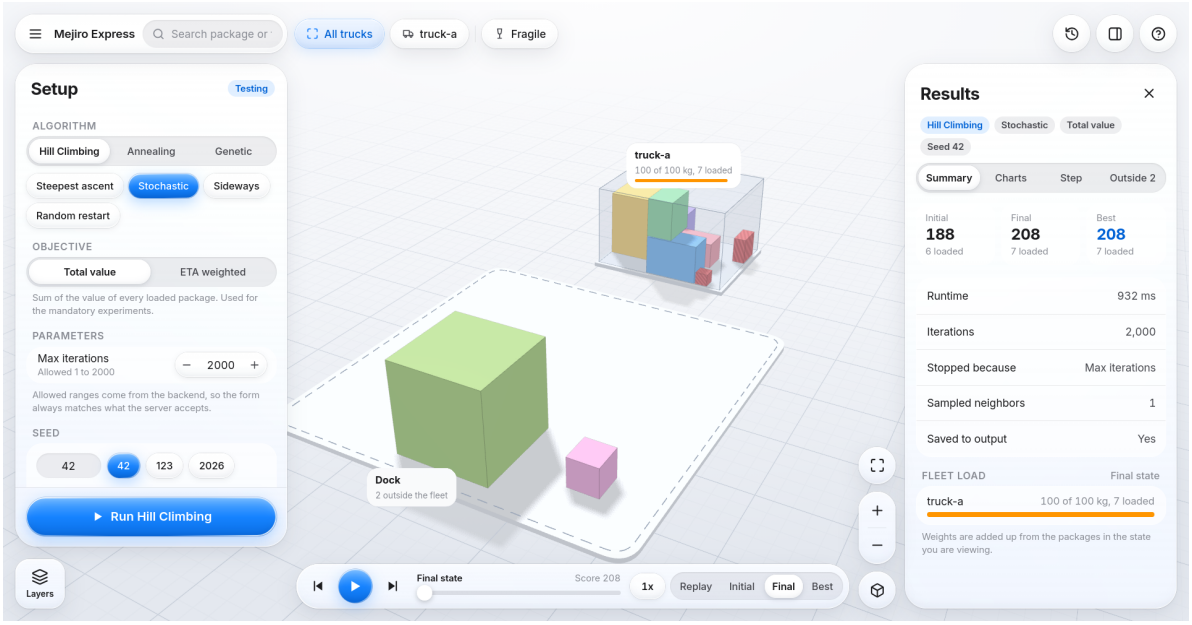
Gambar 9. Bukti Steepest-ascent seed 42: skor akhir 208, skor terbaik 208, runtime 403 ms, dan 2 iterasi.



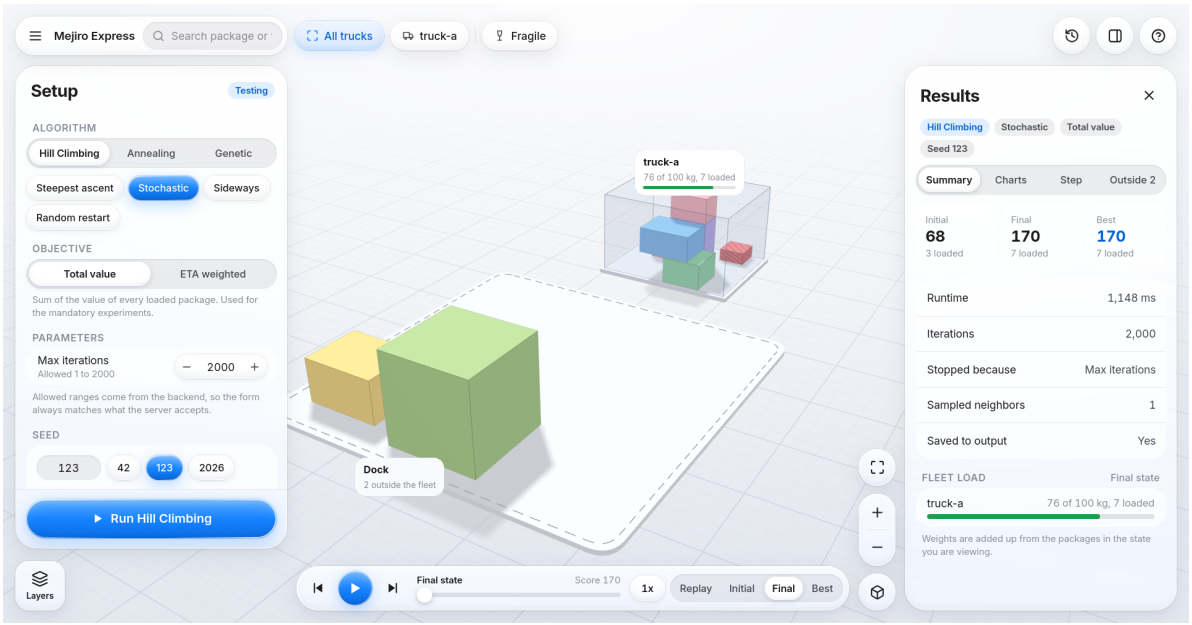
Gambar 10. Bukti Steepest-ascent seed 123: skor akhir 170, skor terbaik 170, runtime 632 ms, dan 5 iterasi.



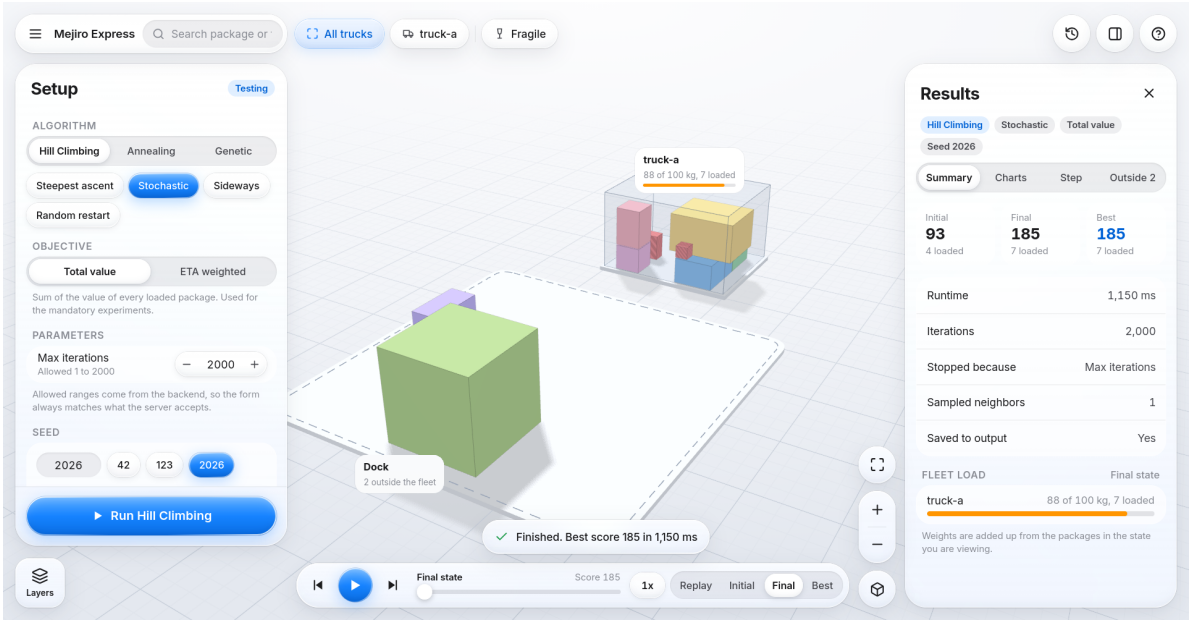
Gambar 11. Bukti Steepest-ascent seed 2026: skor akhir 208, skor terbaik 208, runtime 488 ms, dan 4 iterasi.



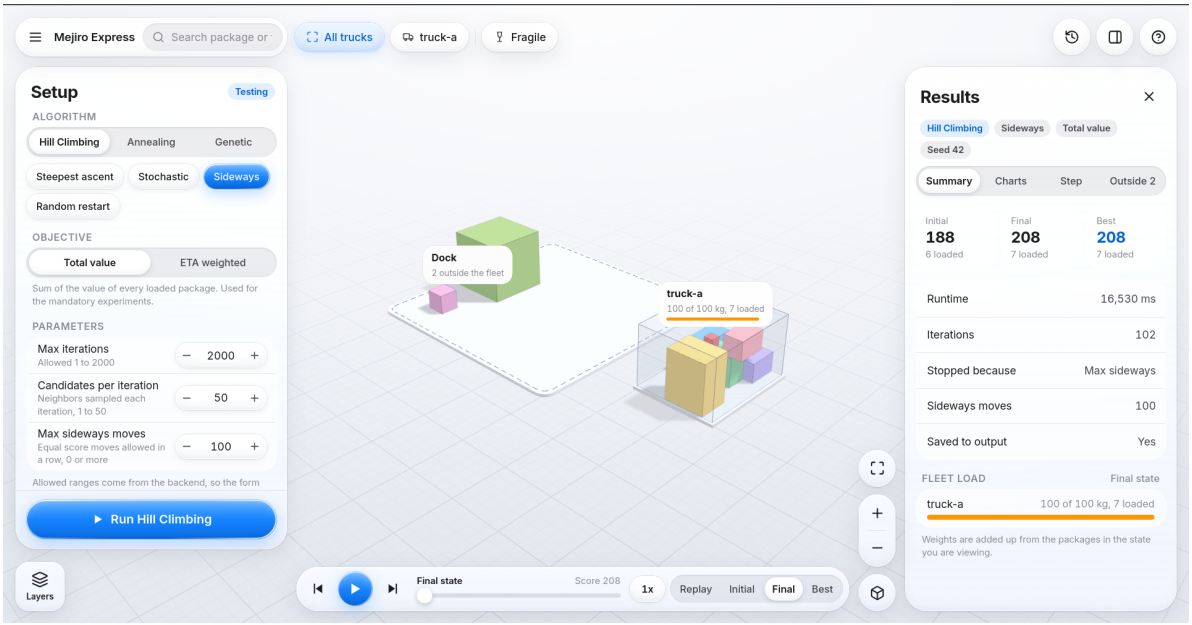
Gambar 12. Bukti Stochastic seed 42: skor akhir 208, skor terbaik 208, runtime 932 ms, dan 2.000 iterasi.



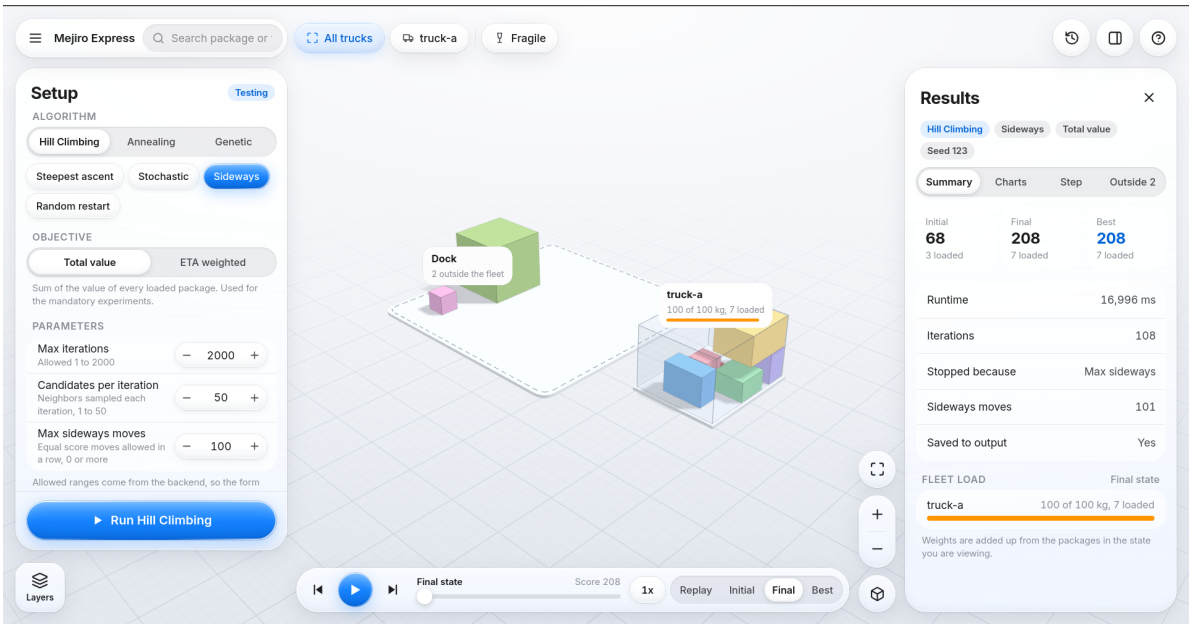
Gambar 13. Bukti Stochastic seed 123: skor akhir 170, skor terbaik 170, runtime 1148 ms, dan 2.000 iterasi.



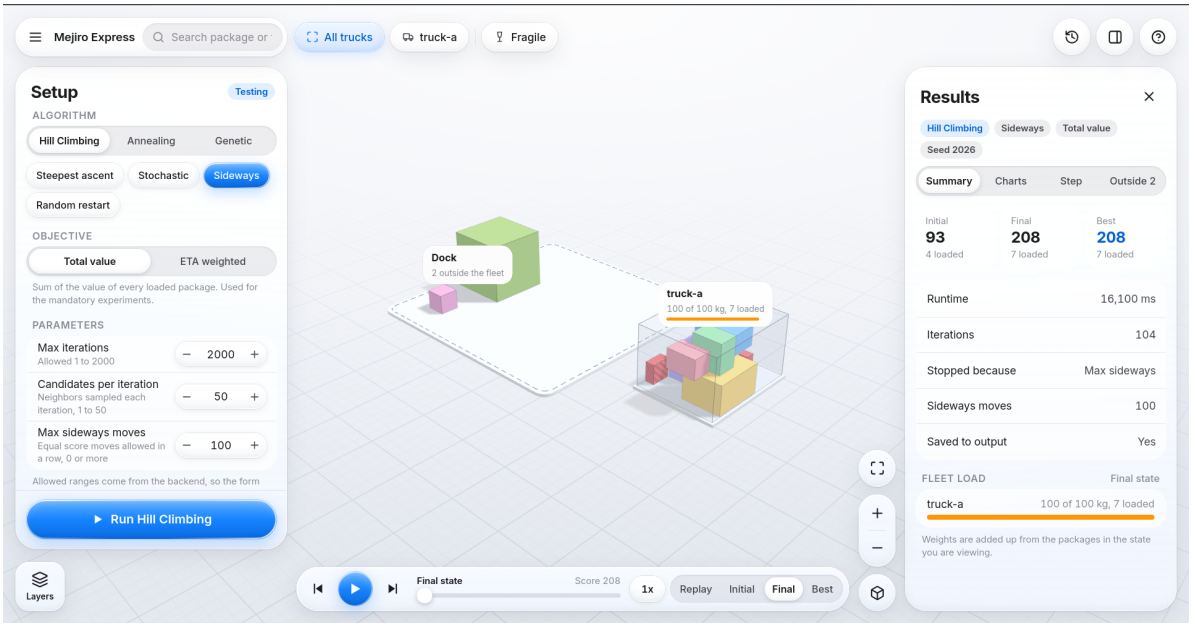
Gambar 14. Bukti Stochastic seed 2026: skor akhir 185, skor terbaik 185, runtime 1150 ms, dan 2.000 iterasi.



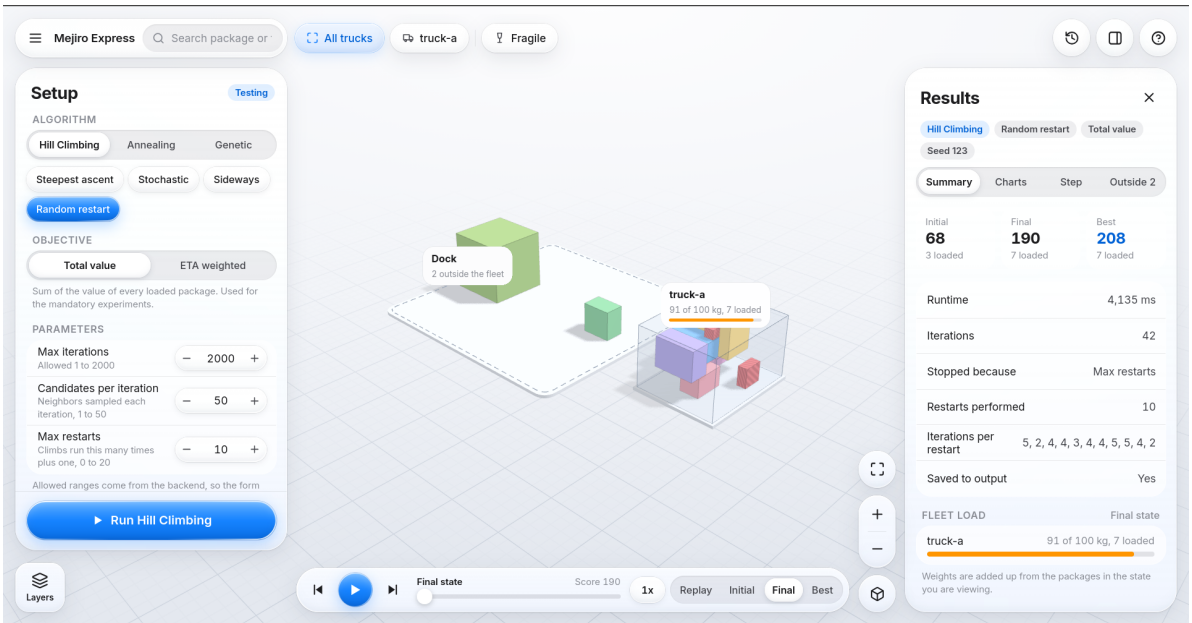
Gambar 15. Bukti Sideways seed 42: skor akhir 208, skor terbaik 208, runtime 16530 ms, dan 102 iterasi.



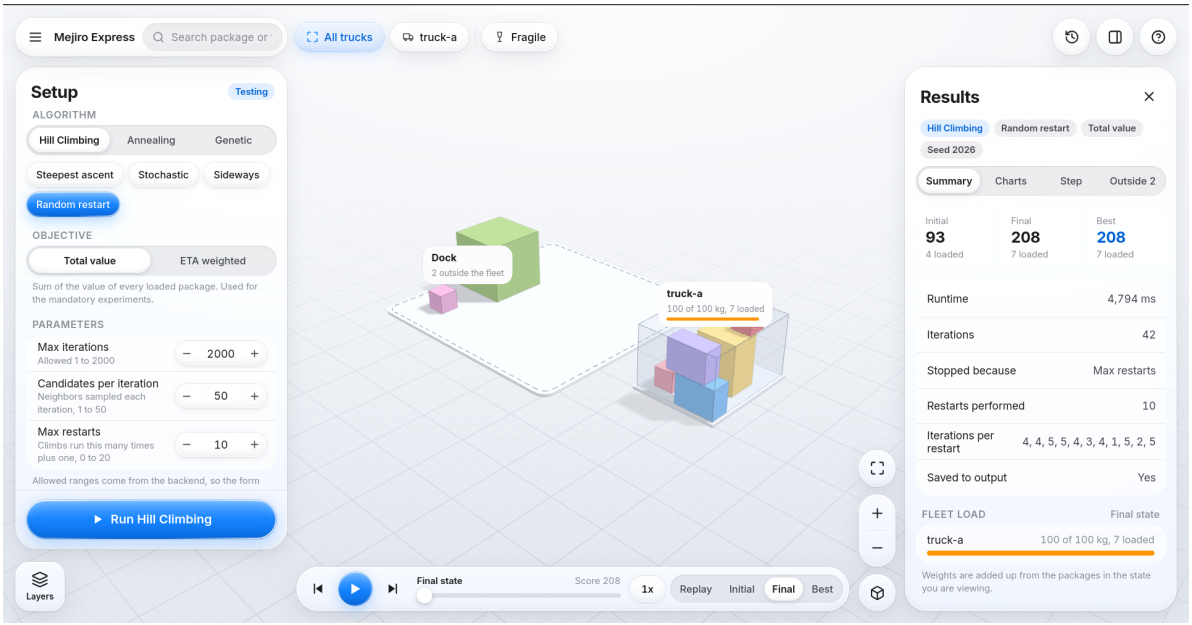
Gambar 16. Bukti Sideways seed 123: skor akhir 208, skor terbaik 208, runtime 16996 ms, dan 108 iterasi.



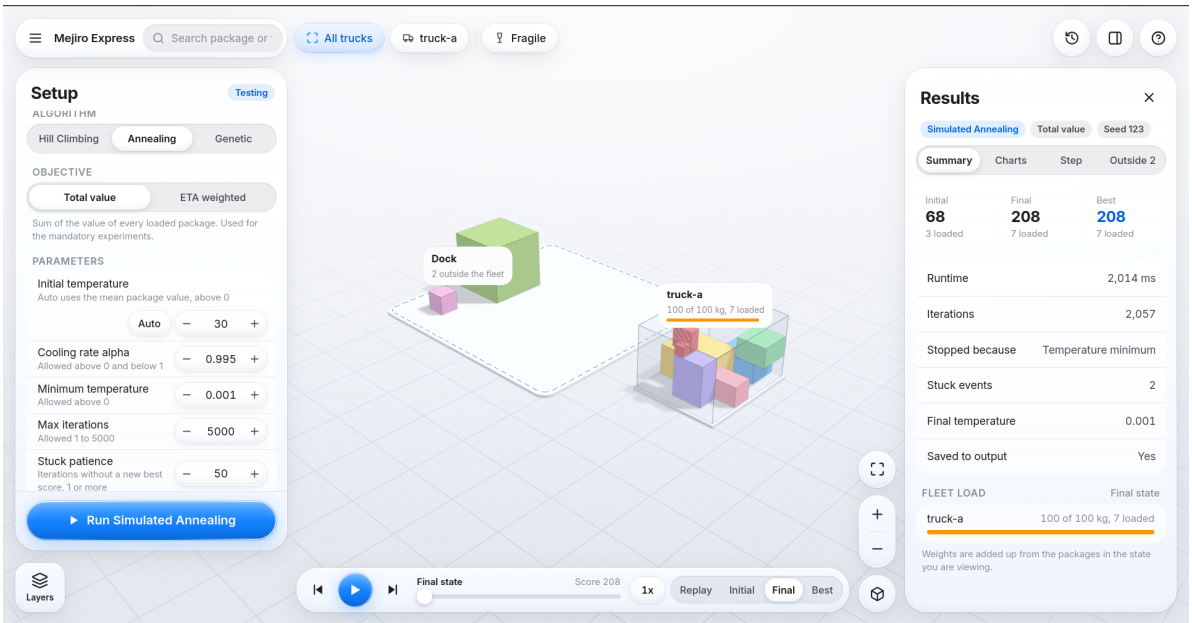
Gambar 17. Bukti Sideways seed 2026: skor akhir 208, skor terbaik 208, runtime 16100 ms, dan 104 iterasi.



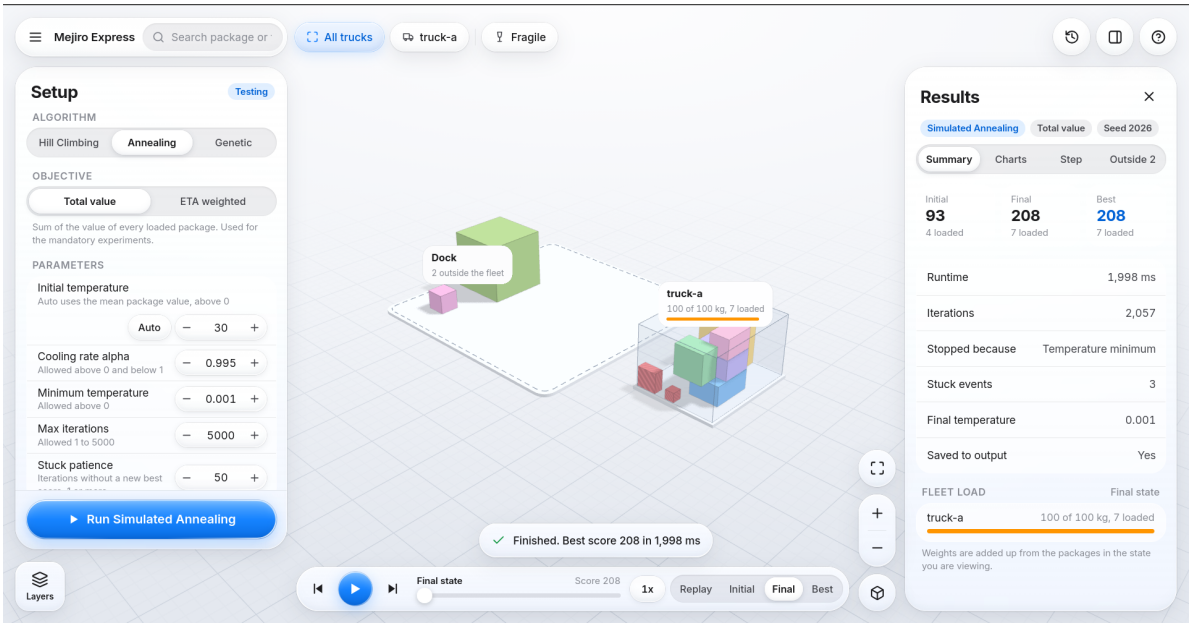
Gambar 18. Bukti Random-restart seed 123: skor akhir 190, skor terbaik 208, runtime 4135 ms, dan 42 iterasi.



Gambar 19. Bukti Random-restart seed 2026: skor akhir 208, skor terbaik 208, runtime 4794 ms, dan 42 iterasi.



Gambar 20. Bukti SA seed 123: skor akhir 208, skor terbaik 208, runtime 2014 ms, dan 2057 iterasi.



Gambar 21. Bukti SA seed 2026: skor akhir 208, skor terbaik 208, runtime 1998 ms, dan 2057 iterasi.